

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

Fakulta elektrotechniky
a komunikačních technologií

BAKALÁŘSKÁ PRÁCE

Brno, 2026

Timotej Komosný



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV AUTOMATIZACE A MĚŘICÍ TECHNIKY

DEPARTMENT OF CONTROL AND INSTRUMENTATION

ŘÍZENÍ BLDC NA PLATFORMĚ STM

BLDC CONTROL ON STM PLATFORM

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Timotej Komosný

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Lukáš Pohl, Ph.D.

BRNO 2026

Bakalářská práce

bakalářský studijní program **Automatizační a měřicí technika**

Ústav automatizace a měřicí techniky

Student: Timotej Komosný

ID: 247397

Ročník: 3

Akademický rok: 2025/26

NÁZEV TÉMATU:

Řízení BLDC na platformě STM

POKYNY PRO VYPRACOVÁNÍ:

- 1) Sestavte matematický model BLDC/PMS motoru v prostředí MATLAB Simulink, implementujte některou z metod řízení momentu
- 2) Nainstalujte vývojové prostředí pro programování mikrokontrolérů STM32, vytvořte konfiguraci pro řízení BLDC/PMSM
- 3) Vytvořte základní aplikaci v jazyce C pro řízení BLDC/PMSM, ekvivalentní aplikaci implementujte v prostředí MATLAB Simulink pomocí knihoven STM
- 4) Proveďte rešerši reálných soustav malé velikosti určených pro výuku řízení
- 5) Navrhněte model jedné z těchto soustav využívající BLDC motor v prostředí MATLAB Simulink
- 6) Realizujte vybranou soustavu z běžně dostupných komponent
- 7) Implementujte řízení motoru na reálné soustavě
- 8) Vyhodnoťte řízení reálné soustavy oproti řízení modelu

DOPORUČENÁ LITERATURA:

NEBORÁK, Ivo; SLÁDEČEK, Václav. Elektrické pohony. Vysoká škola báňská-Technická univerzita Ostrava, 2004.

Termín zadání: 9.2.2026

Termín odevzdání: 27.5.2026

Vedoucí práce: Ing. Lukáš Pohl, Ph.D.

doc. Ing. Miroslav Jirgl, Ph.D.

předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č.40/2009 Sb.

ABSTRAKT

Bakalářská práce se zabývá návrhem a implementací vektorového řízení BLDC motoru a jeho využitím pro stabilizaci rotačního invertovaného kyvadla. V první části práce byl vytvořen matematický model BLDC motoru a implementována momentová regulace na platformě STM32. Následně byl vytvořen nelineární matematický model rotačního invertovaného kyvadla, který byl linearizován v okolí horní rovnovážné polohy.

Pro stabilizaci systému byly navrženy regulátory metodou umístění pólů a metodou LQR společně se stavovým estimátorem. Navržené algoritmy byly implementovány v jazyce C a ověřeny na reálné laboratorní soustavě. Součástí práce je také porovnání simulačních a experimentálních výsledků.

Experimentální výsledky potvrdily úspěšně implementovaný algoritmus vektorového řízení i schopnost stabilizace rotačního invertovaného kyvadla. Dále byly diskutovány rozdíly mezi matematickým modelem a reálnou soustavou.

KLÍČOVÁ SLOVA

BLDC motor, vektorové řízení, PID, STM32, Furuta pendulum, stavová regulace, metoda umístění pólů, lineárně kvadratický regulátor, Luenbergerův estimátor

ABSTRACT

Bachelor thesis focuses on the design and implementation of FOC for the control of BLDC motor and its application to the stabilization of a rotary inverted pendulum. In the first part of the thesis, a mathematical model of the BLDC motor was created and torque control was implemented on the STM32 platform. Subsequently, a nonlinear mathematical model of the rotary inverted pendulum was developed and linearized around the upper equilibrium position.

For system stabilization, controllers based on pole placement and the LQR method were designed together with a state space observer. The proposed algorithms were implemented in the C programming language and verified on a real laboratory setup. The thesis also includes a comparison of simulation and experimental results.

Experimental results confirmed the successful implementation of the FOC algorithm as well as the ability to stabilize the rotary inverted pendulum. Furthermore, differences between the mathematical model and the real system were discussed.

KEYWORDS

BLDC motor, Field Oriented Control, PID, STM32, Furuta pendulum, state feedback control, pole placement, Linear Quadratic Regulator, Luenberger observer

KOMOSNÝ, Timotej. *Řízení BLDC na platformě STM*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2026. Vedoucí práce: Ing. Lukáš Pohl, Ph.D.

Prohlášení autora o původnosti díla

Jméno a příjmení autora: Timotej Komosný
VUT ID autora: 247397
Typ práce: Bakalářská práce
Akademický rok: 2025/26
Téma závěrečné práce: Řízení BLDC na platformě STM

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce. V souvislosti s vytvořením závěrečné práce jsem neporušil zásady a doporučení VUT k využívání generativní umělé inteligence.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
.....
podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Lukáši Pohlovi, Ph.D. za odborné a vstřícné vedení, konzultace, trpělivost a podnětné návrhy k práci.

Obsah

Úvod	13
1 Teoretický rozbor	14
1.1 Typy stejnosměrných motorů	14
1.1.1 Kartáčový motor	14
1.1.2 Bezkartáčový motor	15
1.2 Způsoby řízení BLDC	16
1.2.1 Bezsenzorové řízení	16
1.2.2 Trapézové řízení	16
1.2.3 Vektorové řízení	17
1.3 Rozbor laboratorních úloh teorie řízení	20
1.3.1 Dvouosé invertované kyvadlo	20
1.3.2 Ball and Beam	21
1.3.3 Aerodynamická levitace	22
1.3.4 Furuta pendulum	23
2 Návrh a realizace vektorového řízení BLDC motoru	25
2.1 Simulace řízení BLDC motoru	25
2.1.1 Matematický model BLDC motoru	25
2.1.2 Parametry	26
2.1.3 Vektorové řízení	27
2.2 Implementace řízení BLDC na STM32	29
2.2.1 Vývojová prostředí	29
2.2.2 Použitý hardware	30
2.2.3 Nastavení periférií	33
2.2.4 Implementace algoritmu v jazyce C	37
2.2.5 Implementace algoritmu v Simulinku	39
2.3 Porovnání modelu a reality	42
3 Návrh a realizace řízení systému Furuta pendulum	43
3.1 Realizace rotačního inverzního kyvadla	43
3.2 Dynamický model soustavy	43
3.2.1 Matematický popis	44
3.2.2 Identifikace parametrů	46
3.3 Řídící algoritmus	47
3.3.1 Zpětná stavová regulace	47
3.3.2 Metoda rozmístění pólů	50

3.3.3	Lineární kvadratický regulátor	50
3.3.4	Luenbergerův stavový estimátor	52
3.4	Implementace řízení v jazyce C	53
3.5	Experimentální výsledky a porovnání s modelem	56
Závěr		61
Literatura		62
Seznam symbolů a zkratk		66
Seznam příloh		69
A	Implementace řízení BLDC motoru	70
A.1	Nastavení a povolení pinů pro PWM	70
A.2	Výpočet mechanického a elektrického úhlu	70
A.3	Měření proudů pomocí ADC	71
A.4	Diskrétní PI regulátor	71
A.5	Implementace vektorového řízení	72
A.6	Implementace vektorového řízení v Simulinku	73
B	Furuta Pendulum	74
B.1	Simulace řízení Furuta pendulum s estimátorem	74
B.2	Implementace Luenbergerova pozorovatele	75
B.3	Sestrojené inverzní rotační kyvadlo	76
C	Obsah elektronické přílohy	77

Seznam obrázků

1.1	Stejnoseměrný kartáčový motor	14
1.2	Stejnoseměrný bezkartáčový motor	15
1.3	Trapézové řízení BLDC motoru	17
1.4	Vektorové řízení BLDC motoru	18
1.5	Clarkové transformace	19
1.6	Parkova transformace	20
1.7	Dvouosé invertované kyvadlo s reakčními koly	21
1.8	Ball and Beam	22
1.9	Aerodynamická levitace	23
1.10	Furuta pendulum	24
2.1	Blokové schéma BLDC motoru	25
2.2	Simulační schéma vektorového řízení	27
2.3	Průběh proudů při simulaci roztočení motoru	28
2.4	Nucleo STM32 G474-RE	30
2.5	Výkonová deska IHM16M1	31
2.6	BLDC motor DFA68M024035-E	32
2.7	Módy čítače	34
2.8	Průběh Proudů při běžícím motoru implementace v jazyce C	38
2.9	Vektorové řízení Simulink	41
2.10	Průběh proudů při běžícím motoru implementace Simulink	41
2.11	Přehled průběhů proudů Id	42
3.1	Magnetický senzor AS5147P	44
3.2	Srovnání úhlu kyvadla modelu s realitou	48
3.3	Srovnání úhlu ramene modelu s realitou	48
3.4	Blokové schéma BLDC motoru	49
3.5	Luenbergerův stavový estimátor	53
3.6	Srovnání úhlové rychlosti ramene pomocí estimátoru a derivace	54
3.7	Srovnání úhlové rychlosti kyvadla pomocí estimátoru a derivace	54
3.8	Průběh úhlu θ pomocí návrhové metody umístěním pólů	57
3.9	Průběh úhlu $\dot{\theta}$ pomocí návrhové metody umístěním pólů	57
3.10	Průběh úhlu θ pomocí návrhové metody LQR	58
3.11	Průběh úhlu $\dot{\theta}$ pomocí návrhové metody LQR	58
3.12	Průběh úhlu $\dot{\theta}$ při změně požadovaného úhlu ramene	59
3.13	Průběh úhlu θ při změně požadovaného úhlu ramene	59
3.14	Srovnání úhlu kyvadla modelu s realitou	60
3.15	Srovnání úhlu ramene modelu s realitou	60
A.1	Schéma vektorového řízení	73

B.1	Řízení matematického modelu Furuta pendulum s estimátorem	74
B.2	Laboratorní přípravek Furuta pendulum	76

Seznam tabulek

2.1	Parametry motoru z datasheetu	26
2.2	Dopočtené parametry motoru	26
2.3	Parametry PI regulátoru D složky	28
2.4	Parametry PI regulátoru Q složky	28
2.5	Technické parametry použitého motoru	32
2.6	Přehled použitých pinů	33
2.7	Shrnutí nastavení časovače TIM1	35
2.8	Nastavení TIM1 PWM Generation Channels	35
2.9	Nastavení TIM1 PWM Generation Channels pro Simulink	35
2.10	Shrnutí nastavení časovače TIM8	36
2.11	Přehled nastavení parametrů ADC1	37
2.12	Nastavení programových bloků	40
3.1	Odečtené hodnoty kmitů kyvadla pro identifikaci tření	47
3.2	Přehled použitých pinů	55

Seznam výpisů

A.1	Nastavení portů PWM pro výkonovou desku IHM07M1	70
A.2	Přepočet hodnoty enkodéru na úhel natočení hřídele	70
A.3	Přepočet ADC hodnot na proudy v jednotlivých fázích	71
A.4	Implementace diskrétního PI regulátoru	71
A.5	Regulační smyčka vektorového řízení v D-Q souřadnicovém systému .	72
B.1	Implementace diskrétního Luenbergerova estimátoru v jazyce C . . .	75

Úvod

Vývoj moderních technologií v posledních letech stále více směřuje k autonomním a bezpilotním systémům. Tyto aplikace jsou ve většině případů poháněny elektrickými motory, zejména BLDC motory, které se vyznačují vysokou účinností a dobrými dynamickými vlastnostmi. S rostoucími požadavky na dynamiku, přesnost a energetickou účinnost řízení roste význam pokročilých metod řízení synchronních motorů.

Jednou z nejvýznamnějších metod řízení BLDC motorů je vektorové řízení. Tato metoda umožňuje nezávislé řízení momentotvorné a tokotvorné složky proudu, díky čemuž lze dosáhnout rychlé a přesné regulace momentu motoru a zároveň zvýšit účinnost pohonu.

Kvalitu regulace motoru lze dobře ověřit při řízení nestabilních mechanických soustav. Mezi typické laboratorní úlohy moderní teorie řízení patří například rotační inverzní kyvadlo (Furuta pendulum) nebo Ball and Beam. Tyto systémy se vyznačují nelineární dynamikou, nestabilitou a vysokou citlivostí na parametry regulace, zároveň jsou však dobře popsitelné diferenciálními rovnicemi.

Cílem bakalářské práce je návrh a implementace vektorového řízení BLDC motoru metodou FOC a jeho následné využití pro stabilizaci rotačního invertovaného kyvadla. Součástí práce je vytvoření matematického modelu motoru i samotného kyvadla, návrh stavového regulátoru a estimátoru stavových veličin. Dále je cílem algoritmy realizovat v prostředí MATLAB/Simulink a jazyce C na platformě STM32. Výsledky jednotlivých implementací jsou následně porovnány s matematickými modely a experimentálně vyhodnoceny.

První část práce se zabývá teoretickým rozbořem elektrických motorů se zaměřením na BLDC motor a jeho metody řízení. Následuje část věnovaná vytvoření matematického modelu motoru a implementaci řízení metodou FOC na platformě STM32. Třetí část práce je zaměřena na konstrukci rotačního invertovaného kyvadla, vytvoření jeho matematického modelu a návrh metod stavového řízení. V závěru práce jsou prezentovány experimentální výsledky a jejich porovnání se simulací.

1 Teoretický rozbor

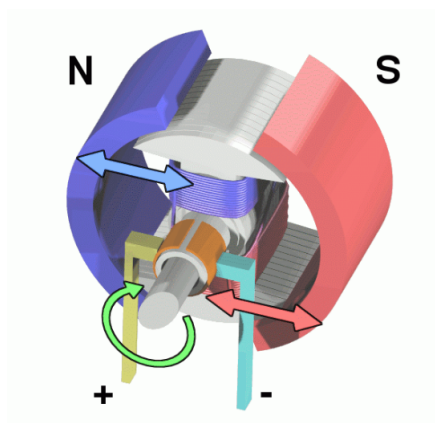
1.1 Typy stejnosměrných motorů

1.1.1 Kartáčový motor

Stejnosměrné motory, dále jen DC motory, jsou elektrické motory napájené stejnosměrným napětím. Používají se v aplikacích, kde je potřeba silný točivý moment a změna rychlosti otáčení. V praxi se využívají v tiskárnách, jeřábech nebo lisovnách, všude tam, kde je potřeba řízení rychlosti.

DC motor má dvě části: stator a rotor. Stator je tvořen permanentními magnety nebo budícím vinutím, které vytváří magnetické pole. Rotor je tvořen válcovou strukturou s děrami, okolo kterých je navinut měděný vodič. Konce vodiče jsou připojeny ke komutátoru, který je umístěn na stejné hřídeli jako rotující část; tento celek je označován jako kotva. Napětí se přivádí na rotor pomocí sběracích kartáčů, které se dotýkají jednotlivých segmentů na komutátoru. Ten přepíná vinutí rotoru a zajišťuje plynulý průběh otáčení. Proto jsou tyto motory označovány jako kartáčové DC motory („brushed DC motor“).[1]

Nevýhodou kartáčového motoru je tření kartáčů o komutátorové plochy, čímž dochází k mechanickému namáhání materiálu, dokud se kartáče úplně opotřebují a ztratí kontakt s komutátorem. Větší motory jsou uzpůsobeny pro výměnu kartáčů, to ale vyžaduje častou údržbu motorů.[2]



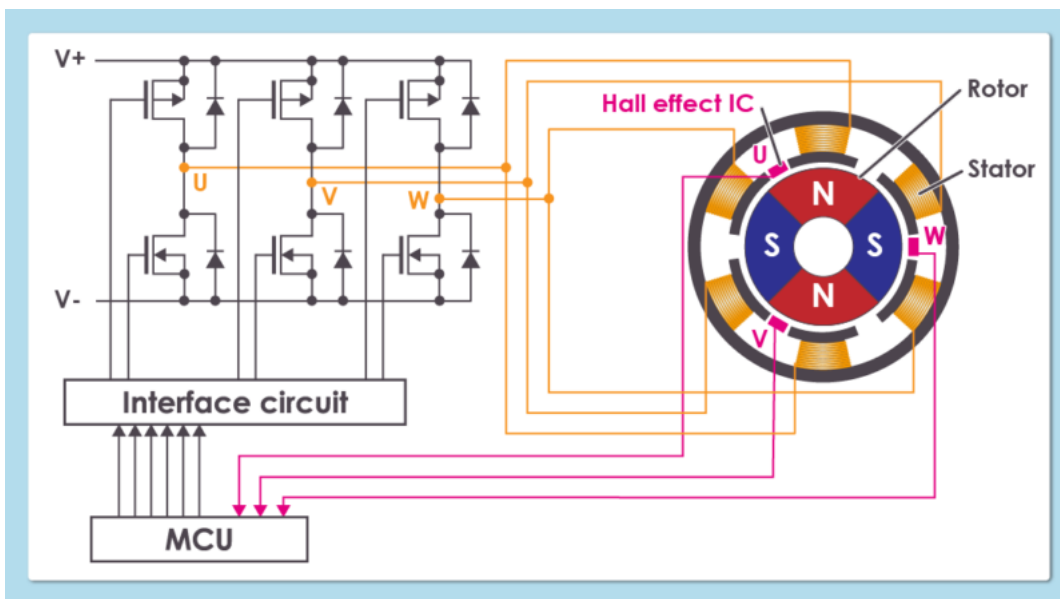
Obr. 1.1: Stejnosměrný kartáčový motor[3]

1.1.2 Bezkartáčový motor

BLDC (brushless DC motor) jak název napovídá, je stejnosměrný motor, který nemá kartáče, takže není potřeba ani komutátor. Odpadá tím problém s mechanickým třením kartáčů a nutnost časté údržby.

Konstrukce BLDC je odlišná od DC motoru. Rotor je tvořen permanentními magnety, existují dva typy rotoru. Vnitřní („inrunner“) má rotor uvnitř statoru a vnější („outrunner“) má rotor na vnější části. Stator je tvořen cívkami, které jsou rozděleny do třech fází a většinou zapojeny do hvězdy. Při postupném spínání jednotlivých fází se vytváří točivé magnetické pole. BLDC motor tedy vyžaduje střídavé napětí, ale je napájen stejnosměrným. To proto, že je motoru předřazen střídač, většinou H-můstek tvořený IGBT tranzistory. Tomu je předáván PWM signál, který je přetvořen na střídavé napětí a připojen na příslušné fáze. Konstrukce motoru je tedy podobná DC motoru, avšak chování je podobné asynchronnímu motoru s permanentními magnety a to díky spínání jednotlivých fází sinusovým signálem. [2][4]

Pro správné řízení BLDC motoru je potřeba znalost polohy rotoru. Využívají se různé snímače, jako jsou Hallovy sondy, inkrementální čítače, resolvery nebo magnetické snímače. [4][5]



Obr. 1.2: Stejnosměrný bezkartáčový motor ovládán střídačem s mikroprocesorem [5]

1.2 Způsoby řízení BLDC

1.2.1 Bezsenzorové řízení

Jak už bylo zmíněno, BLDC motor potřebuje pro správný běh znalost polohy, aby mohl ve správný čas indukovat napětí na cívky statoru (elektrická komutace). Čas pro správnou komutaci se dá určit i jiným způsobem, a to pomocí měření „back EMF“. Při běhu BLDC motoru vzniká BEMF a jeho velikost odpovídá specifickému úhlu rotoru; díky tomu je možné určit polohu rotoru. BEMF může mít sinusový nebo trapézový průběh, v závislosti na konstrukci motoru.

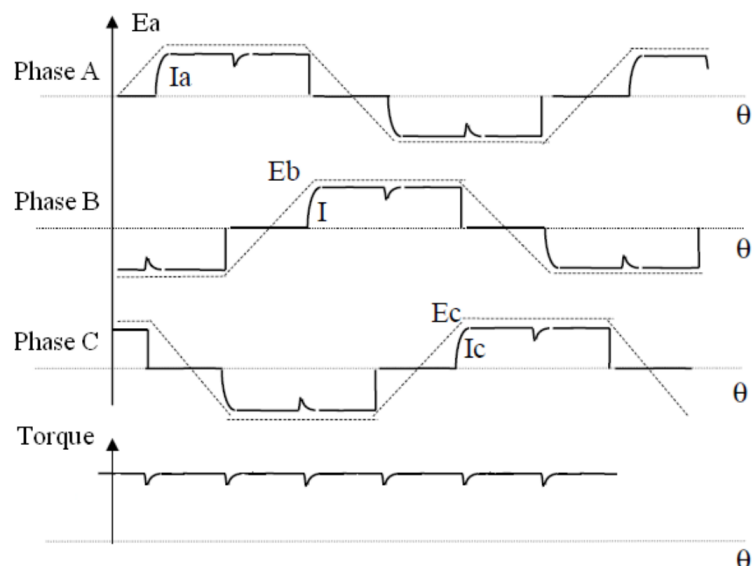
Při bezsenzorovém řízení je důležitá počáteční pozice rotoru. Používají se různé metody pro určení základní pozice, jako například „Align and go“. Rotor je zarovnán do určené pozice pomocí aktivace dvou fází statoru, které vytvoří magnetické pole, a to rotor zarovná. Další metodou je řízení v otevřené smyčce, kde je postupně zvětšována frekvence napájecího signálu. Tato metoda umožňuje odhad počáteční polohy rotoru za postupného rozběhu.

Řízení BLDC motoru bez použití senzorů snižuje náklady a konstrukční složitost motoru. Na druhou stranu je zapotřebí výkonný procesor s velkou pamětí a výpočetním výkonem. Tato metoda je aplikovatelná i na trapézové a vektorové řízení[6][7]

1.2.2 Trapézové řízení

„Trapezoidal control“ neboli trapézové řízení BLDC motoru je nejjednodušší metoda, jak řídit BLDC motor. Trapézové řízení probíhá v šesti krocích, proto je také někdy označováno jako „six-step commutation“.

Fáze statoru jsou postupně buzeny stejnosměrným proudem. Vždy je jedna fáze v nule a dvě další v záporné a kladné hodnotě. Tím se úhel vždy změní o 60 stupňů a po šesti krocích je tedy rotor otočen o 360 stupňů. Pro správné načasování komutace se využívají senzory polohy, jako například Hallovy sondy. Postupné změny proudů vytvářejí trapézový průběh, viz obrázek 1.3.[8][2]

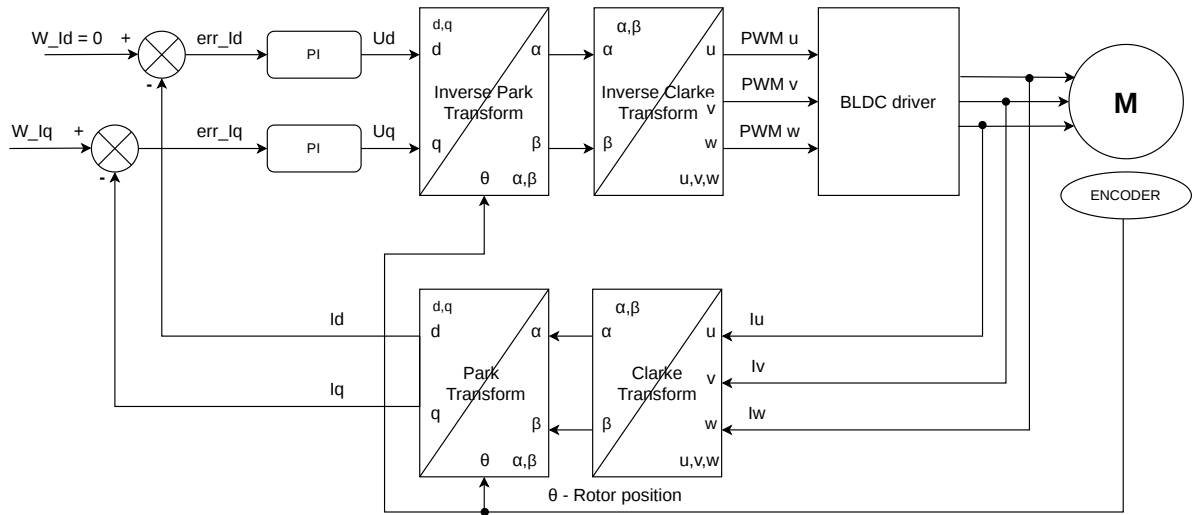


Obr. 1.3: Trapézové řízení BLDC motoru[8]

1.2.3 Vektorové řízení

Vektorové řízení, neboli anglicky FOC, je pokročilejší metodou řízení BLDC motoru. Tato metoda nabízí nejlepší točivý moment pro daný systém. Toho je dosaženo díky rozdělení točivého momentu a magnetického toku na separátní složky, a tedy možnosti řídit pouze točivý moment. Jednotlivé složky jsou získány pomocí matematických operací, které musí být provedeny v reálném čase. Kvůli tomu je pro FOC potřeba výkonný mikrokontrolér.

Při FOC jsou snímány proudy protékající fázemi a je snímána pozice rotoru. Ve všech třech fázích jsou měřeny proudy, ty jsou od sebe posunuty o 120° . Proudů je potřeba přetransformovat z třífázové soustavy I_A, I_B, I_C do dvoufázového systému I_{α}, I_{β} pomocí Clarkovy transformace. Dále se transformuje do d, q souřadnicového systému pomocí Parkovy transformace, ta vyžaduje informaci o poloze rotoru. V tomto systému je provedena regulace jednotlivých složek I_d, I_q pomocí PI regulátoru. Tím vzniknou veličiny akčního zásahu U_d, U_q , ty jsou přetransformovány zpět do třífázové soustavy pomocí zpětné Clarkovy a Parkovy transformace. Tyto napětí jsou předány pomocí PWM do střídače, který vytváří sinusový signál pro jednotlivé fáze motoru. Celé blokové schéma je na obrázku 1.4 [9][2]



Obr. 1.4: Vektorové řízení BLDC motoru

Clarkové transformace

Pro převod z třífázového systému na dvoufázový ortogonální systém je použita transformace Edith Clarkové. Vektory po transformaci jsou stále časově závislé. [4][9]

$$i_{\alpha} = \frac{1}{3} \cdot (2i_a - i_b - i_c) \quad (1.1)$$

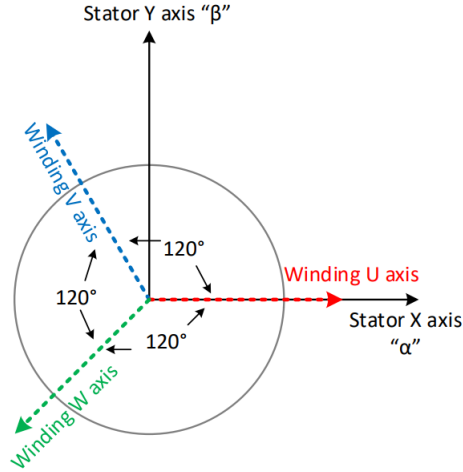
$$i_{\beta} = \frac{\sqrt{3}}{3} \cdot (i_b - i_c) \quad (1.2)$$

Pro transformaci z alfa beta systému do třífázového se používá zpětná transformace Clarkové. [4][9]

$$u_a = u_{\alpha} \quad (1.3)$$

$$u_b = -\frac{1}{2}i_{\alpha} + \frac{\sqrt{3}}{2}i_{\beta} \quad (1.4)$$

$$u_c = -\frac{1}{2}i_{\alpha} - \frac{\sqrt{3}}{2}i_{\beta} \quad (1.5)$$



Obr. 1.5: Clarkové transformace[10]

Parkova transformace

Parkova transformace se používá pro převod z ortogonálního dvoufázového systému (alfa - beta) na referenční rotační systém(d, q). Kde složka d je tokotvorná a momentotvorná složka q. Pro transformaci je nutná poloha rotoru vyjádřena úhlem φ . Vektory po transformaci jsou časově nezávislé.[4][9]

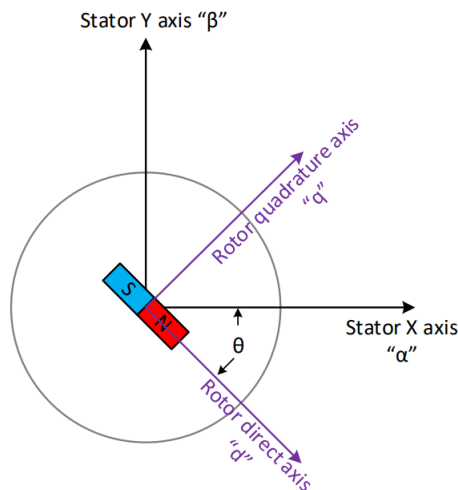
$$i_d = i_\beta \sin(\varphi) + i_\alpha \cos(\varphi) \quad (1.6)$$

$$i_q = i_\beta \cos(\varphi) - i_\alpha \sin(\varphi) \quad (1.7)$$

Pro transformaci z referenčního rotačního systému d-q do ortogonálního dvoufázového souřadnicového systému se používá zpětná Parkova transformace.[4][9]

$$u_\alpha = u_d \cos(\varphi) - u_q \sin(\varphi) \quad (1.8)$$

$$u_\beta = u_d \sin(\varphi) + u_q \cos(\varphi) \quad (1.9)$$



Obr. 1.6: Parkova transformace[10]

1.3 Rozbor laboratorních úloh teorie řízení

Tato kapitola pojednává o vybraných reálných soustavách malých rozměrů, které se používají pro výuku teorie řízení. Jednotlivé úlohy jsou popsány z hlediska jejich principu, vlastností a vhodného návrhu regulace. Cílem kapitoly je ukázat rozmanitost úloh a vybrat vhodnou úlohu pro další zpracování, kterou je v této práci „Furuta pendulum“.

1.3.1 Dvoustupňové invertované kyvadlo

Dvoustupňové invertované kyvadlo s reakčními koly je také někdy označováno jako („self-balancing stick“). Patří do skupiny úloh řízení inverzního kyvadla, přičemž se liší způsobem stabilizace. Stabilizace je dosažena pomocí momentu hybnosti rotujících kol. Úloha využívá Newtonův třetí zákon: pro každou akci existuje stejná a opačná reakce. Motor aplikuje točivý moment na kolo, čímž vzniká reakční moment působící na tělo kyvadla. Reakční kola se například používají k orientaci družic ve vesmíru bez použití raketového paliva.

V článku jsou odvozeny nelineární rovnice pomocí Lagrangeovy metody, a následně byla provedena linearizace v okolí pracovního bodu. Řízení bylo navrženo pomocí metody umístění pólů, protože chyběla měření některých stavů. Tato soustava je blízká tématu této práce, neboť vyžaduje přesné a rychlé řízení motorů, čehož lze dosáhnout právě pomocí algoritmu FOC. [11]



Obr. 1.7: Dvouosé invertované kyvadlo s reakčními koly[12]

1.3.2 Ball and Beam

Systém kulička-nosník („Ball and Beam“) představuje klasickou úlohu v teorii řízení a patří do skupiny nestabilních systémů s poddimenzovaným řízením („underactuated“). Tedy systémy, které mají více stupňů volnosti než akčních zásahů. Stabilizace kuličky je dosažena změnou úhlu náklonu nosníku, čímž dochází ke změně gravitační síly působící na kuličku.

Základní princip spočívá v přeměně rotačního momentu motoru na přímočarý pohyb kuličky. Systém vyžaduje měření úhlu natočení motoru a senzor pro měření polohy kuličky. Tato úloha je považována za nejvíce praktickou úlohu pro studium nelineárních systémů.

Nelineární pohybové rovnice systému jsou odvozeny pomocí Lagrangeovy metody a následně je provedena jejich linearizace v okolí vodorovné rovnovážné polohy. Vzhledem k nelineární vazbě mezi nosníkem a kuličkou je vhodné použít LQR nebo PID regulátor. K odhadu rychlosti kuličky, která není přímo měřena, je možné použít Kalmanův filtr. [13][14]



Obr. 1.8: Ball and Beam[13]

1.3.3 Aerodynamická levitace

Úloha aerodynamické levitace kuličky je založena na principu vznášení objektu v proudu vzduchu, který je v tomto případě generován ventilátorem uvnitř tubusu. Aplikace nalezneme tam, kde je nežádoucí kontakt dvou objektů, například tam, kde je potřeba zabránit oxidaci nebo krystalizaci na povrchu materiálu.

Základní fyzikální princip využívá rovnováhu sil působících na kuličku: gravitační síly a aerodynamické odporové síly. Odporová síla závisí na druhé mocnině relativní rychlosti kuličky vůči proudu vzduchu, což do systému vnáší nelinearitu. Soustava je zajímavá svou pomalejší dynamikou oproti magnetické levitaci. To umožňuje použití delší periody vzorkování a výpočetně náročnějších algoritmů.

Úloha je vhodná pro výuku řízení díky možnosti vyjádření nelineárních rovnic pomocí Lagrangeovy metody a linearizace. Díky tomu lze na modelu testovat různé druhy regulátorů společně s estimátory pro odhad neměřených stavů. [16]



Obr. 1.9: Aerodynamická levitace[16]

1.3.4 Furuta pendulum

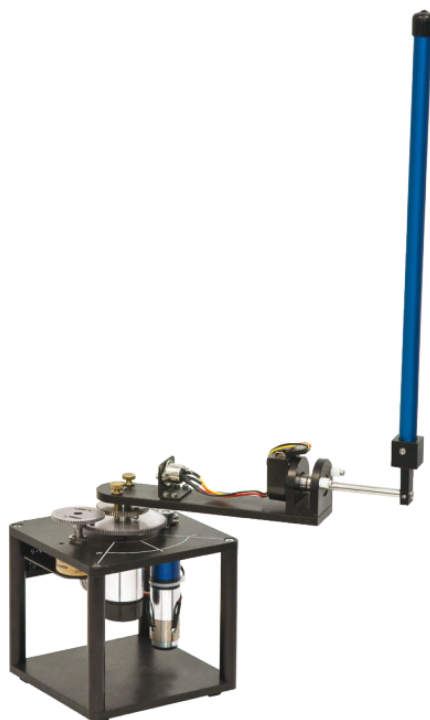
Rotační invertované kyvadlo, známé jako Furuta pendulum, bylo vyvinuto na Tokijském technologickém institutu Katsuhisou Furutou a jeho kolegy. Společně s klasickým inverzním kyvadlem na vozíku je velice oblíbenou problematikou v řízení nestabilních systémů. Systém se skládá z horizontálního ramene poháněného motorem, na jehož konci je uchyceno volně otočné kyvadlo kmitající ve vertikální rovině.

Soustava má dva stupně volnosti, ale pouze jeden motor, což z ní činí poddimenzovaný systém. Při rotaci ramene vznikají odstředivé a Coriolisovy síly, což ze systému činí silně nelineární a stavově provázanou soustavu. K řiditelnosti je vyžadováno měření úhlu natočení motoru a úhlu náklonu ramene. Stabilizace v horní rovnovážné poloze vyžaduje přesné a plynulé řízení momentu motoru, čehož lze dosáhnout pomocí algoritmu FOC.

Aplikací Lagrangeovy metody a následnou linearizací v okolí pracovní polohy lze získat stavový popis systému se čtyřmi stavy (dva úhly a jejich rychlosti). Úhlové rychlosti musí být dopočítány z úhlů, čímž vzniká šum, a tím se regulace pomocí PID regulátoru stává problematickou. Z tohoto důvodu se pro řízení často využívá zpětná stavová regulace („Feed-back control“), například pomocí metod rozložení pólů nebo LQR. Pro získání dvou chybějících stavů lze využít stavové estimace („Observer“), jako je Luenbergerův pozorovatel.

Specifikem řízení Furuta pendulum je nutnost využití dvou odlišných pracovních režimů. Prvním je tzv. „Swing-up“ algoritmus, který zajistí rozhoupání kyvadla z dolní polohy do nestabilní horní polohy. V této fázi se řízení přepne na stabilizaci,

například pomocí LQR. Pro swing-up se často využívá nelineární řízení založené na Ljapunovské stabilitě („Energy-based control“). Metodou swing-up se tato práce nezabývá a je zaměřena na stabilizaci ramena v horní poloze.[17][18][19]



Obr. 1.10: Furuta pendulum[19]

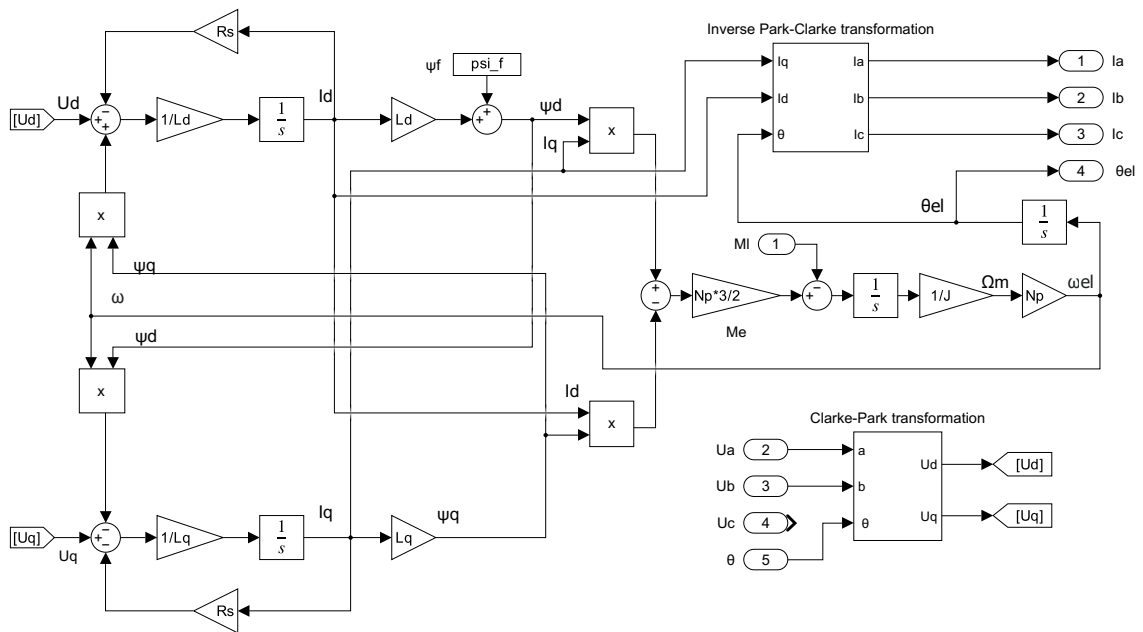
2 Návrh a realizace vektorového řízení BLDC motoru

2.1 Simulace řízení BLDC motoru

2.1.1 Matematický model BLDC motoru

BLDC motor je vhodné simulovat v d, q souřadnicovém systému. Toho je dosaženo pomocí Clarkovy a Parkovy transformace. Díky transformacím třífázových střídavých veličin do rotující soustavy vzniknou veličiny téměř konstantní v čase, čímž se výrazně zjednoduší matematický popis motoru. A zároveň ve stejném souřadnicovém systému probíhá regulace proudů pomocí algoritmu FOC.

Blokové schéma modelu bylo sestaveno na základě matematických rovnic uvedených v [4].



Obr. 2.1: Blokové schéma BLDC motoru

Horní větev modelu popisuje dynamiku tokotvorné složky proudu I_d a spodní větev popisuje momentotvornou složku I_q . Rozdíl mezi napětím, odporem vinutí R_s a indukovaným napětím je podělen indukčností a následně integrován; tím vznikají proudy I_d a I_q . Proudů jsou vynásobeny indukčnostmi, k I_d je potřeba přičíst tok od permanentních magnetů ψ_f , tím vznikají magnetické spřažené toky ψ_d a ψ_q . Elektromagnetický moment stroje M_e je dán vektorovým součinem toků a proudů,

vynásobeným počtem pólových párů. Výsledný moment po odečtení zátěžného momentu M_L je integrován a dělen momentem setrvačnosti. Tím vzniká mechanická úhlová rychlost motoru Ω_m . Vynásobením počtem pólových dvojic je Ω_m převedena na elektrickou úhlovou rychlost ω_{el} . Po integraci se z úhlové rychlosti stává úhel θ_{el} , ten je dále využit pro transformace. Pro získání proudů v reálných fázích I_a, I_b, I_c jsou složky I_d a I_q převedeny pomocí zpětné Parkovy a Clarkové transformace.

2.1.2 Parametry

Potřebné parametry pro simulaci byly získány z datasheetu motoru.

Tab. 2.1: Parametry motoru z datasheetu

R_s	0.25 [Ω]	N_p	14[-]
L_d	0.22 [mH]	ψ_f	0.0048[Wb]
L_q	0.22 [mH]	J	0.0001 [$kg \cdot m^2$]

Při porovnání s realitou byly pro model použity parametry z tabulky 2.2. Odpor R_s a indukčnost L_d a L_q byly získány z přechodové charakteristiky reálného motoru. Magnetický tok ψ_f byl dopočítán pomocí rovnic uvedených v [4], přes momentovou konstantu. Moment setrvačnosti byl určen při modelování Furuta pendula.

$$M_e = \frac{3}{2}p\psi_f i_q \quad (2.1)$$

$$\psi_f = \frac{K_t}{\frac{3}{2} \cdot p} \quad (2.2)$$

Tab. 2.2: Dopočtené parametry motoru

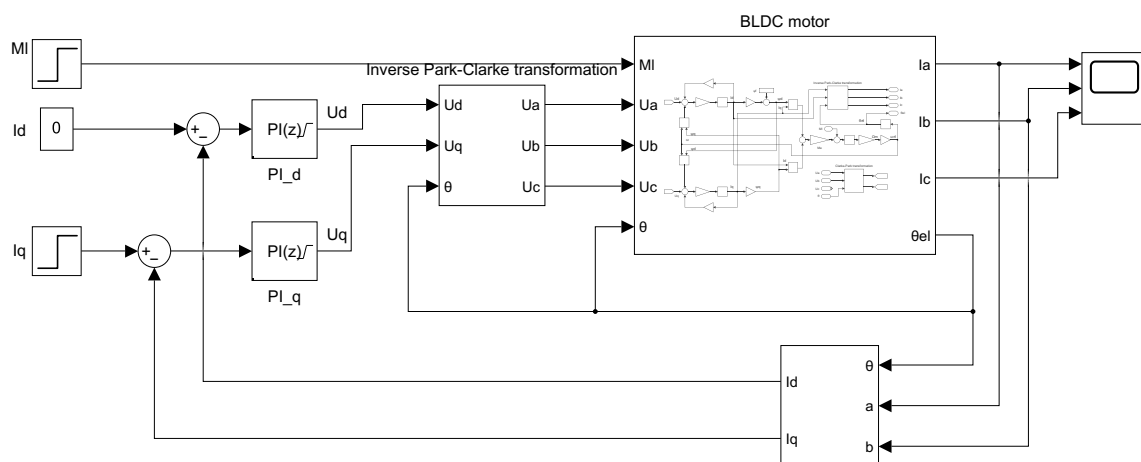
R_s	1.8 [Ω]	N_p	14 [-]
L_d	0.75 [mH]	ψ_f	0.0048 [Wb]
L_q	0.75 [mH]	J	0.0009 [$kg \cdot m^2$]

Rozdíl v hodnotách je nejspíš zapříčiněn velkým odporem kabelů, které propojují motor a výkonovou desku.

2.1.3 Vektorové řízení

Na modelu BLDC motoru bylo aplikováno vektorové řízení. Z modelu vystupují třífázové proudy a elektrický úhel. Proudů jsou přetransformovány do D-Q souřadnicového systému, ve kterém je vypočítána regulační odchylka, a pomocí PI regulátorů je vytvořeno napětí U_q a U_d . Tyto napětí po transformaci řídí otáčení motoru. Do modelu dále vstupuje úhel pro transformace a zátěžový moment pro simulaci zátěže.

Implementovaná proudová/momentová regulační smyčka je jádrem FOC. Dále se algoritmus dá rozvinout o rychlostní a polohovou smyčku. Protože je momentová smyčka první a ostatní na ni závisí, musí být také nejrychlejší. Rychlost regulační smyčky zajišťuje vzorkovací frekvence a správné nastavení PI regulátorů.



Obr. 2.2: Simulační schéma vektorového řízení

Aby matematický model odpovídal reálnému systému, byly pro simulaci zvoleny stejné parametry regulátorů, jaké byly implementovány v regulační smyčce fyzické soustavy. Při praktickém ladění parametrů P a I se osvědčil postup nejprve nastavit regulátor tokotvorné složky. Díky jejím vlastnostem se při změně proudu motor nerozběhne, čímž se stává nastavování parametrů jednodušším. Jednotlivá zesílení byla přenesena do regulátoru momentotvorné složky a další iterací se doladila. Vysoké hodnoty zesílení I složky obou regulátorů jsou dány malou vzorkovací periodou $50 \mu s$, která vyžaduje větší integrační složku pro eliminaci odchylky.

Tab. 2.3: Parametry PI regulátoru D složky

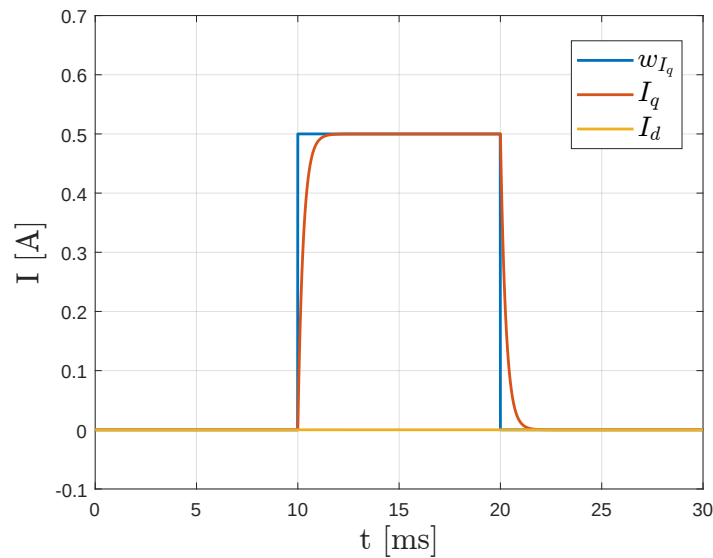
Složka Reguátoru	Zesílení
K_P	2
K_I	4500

Tab. 2.4: Parametry PI regulátoru Q složky

Složka Reguátoru	Zesílení
K_P	2.7
K_I	6100

Pro zajištění správného chování FOC algoritmu je potřeba ověřit, zda proudy dosáhnou požadované hodnoty. Hlavními sledovanými parametry jsou doba náběhu proudu na žádanou hodnotu a minimalizace překmitu, který by mohl vést k nestabilitě regulace. Ověření je potřeba provést pro obě složky, protože I_d může být zdánlivě nastaveno správně, ale při rozběhnutí motoru regulátor nestačí veličinu vyregulovat, tím nebude reakce dostatečně rychlá a část proudu pronikne z I_q do složky I_d .

Rozběhnutí motoru lze simulovat pomocí nastavení požadované hodnoty proudu I_q .



Obr. 2.3: Průběh proudů při simulaci roztočení motoru

Obrázek 2.3 zachycuje odezvu proudů I_q a I_d na požadovanou hodnotu w_{I_q} . Si-

mulace modelu je bez dopravního zpoždění, proto transformace dostávají přesné informace o úhlu. Transformace tedy správně řídí momentotvornou složku a I_d zůstává na nule. Náběh proudu na požadovanou hodnotu trvá přibližně 2 ms, což je dostatečně rychlá reakce pro správné fungování FOC. Průběh proudu I_d je pro větší přehlednost vyneseno v grafu na obrázku 2.11 společně s ostatními proudy.

2.2 Implementace řízení BLDC na STM32

2.2.1 Vývojová prostředí

Matlab a Simulink

MATLAB je rozsáhlé programovací prostředí uzpůsobené pro práci s daty, maticemi, filtry, simulacemi, modelováním. Díky rozsáhlým knihovnám umožňuje jednoduchý zápis jinak složitých operací (vysokoúrovňový programovací jazyk). Umožňuje efektivně vytvářet matematické modely systémů, simulovat jejich chování a analyzovat výsledky.

Simulink je jedním z rozšíření (toolbox) k Matlabu, které umožňuje blokově modelovat dynamické systémy, testovat řídicí algoritmy nebo simulovat fyzikální procesy. Společně s MATLAB představují účinný nástroj pro modelování a simulaci. Pomocí rozšíření embedded coder support package for STMicroelectronics STM32 Processors, simulink umožňuje vývoj aplikací pro mikrokontroléry STM32. [20]

CubeMX

STM32CubeMX je grafický konfigurační nástroj od STMicroelectronics, navržený pro zjednodušení a urychlení vývoje aplikací pro mikrokontroléry STM32. Užitečné je i přehledné prostředí, správa knihoven a nabídka příkladů projektů. Poskytuje jednoduchou konfiguraci mikroprocesoru a jeho periférií. Nastavenou konfiguraci převádí do jazyka C a vytváří nebo doplňuje existující projekt. Programátor musí pouze dávat pozor, aby svůj kód psal do správných částí projektu. [21]

CubeIDE

STM32CubeIDE je pokročilá C/C++ vývojová platforma pro vytváření aplikací na STM32. Obsahuje editor kódu, debugger a nástroje pro konfiguraci projektů v jednom prostředí. Je úzce spojen s CubeMX pro efektivní vývoj aplikací. [22]

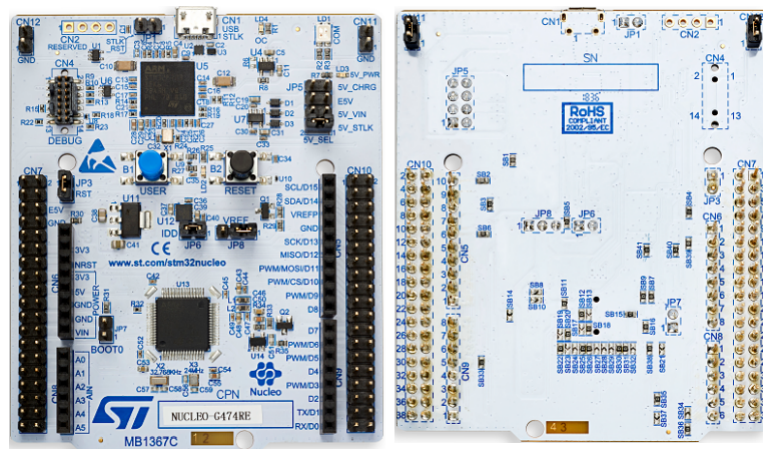
CubeMonitor

STM32CubeMonitor je nástroj pro monitorování a analýzu běžících aplikací na mikrokontrolérech STM32 v reálném čase. Umožňuje vizualizaci dat pomocí grafů nebo tabulek. Po připojení k ST-linku začne komunikace přes UART. ST-Link umožňuje připojení pouze jednoho zařízení v daný okamžik, proto lze pozorovat data pouze v jednom prostředí. Díky funkci snapshot je možné sledovat detailní průběh měřené veličiny při vysoké vzorkovací frekvenci. Naměřené průběhy veličin lze uložit do souboru a dále s nimi pracovat například v Matlabu.[23]

2.2.2 Použitý hardware

Nucleo STM32G474-RE

Mikrokontrolér obsahuje jádro založené na ARM Cortex-M4 s FPU, což umožňuje výpočty s plovoucí desetinou čárkou. Taktovací frekvence jádra je 170 MHz. Debugger a programátor jsou umístěny přímo na desce (ST-link), takže nejsou potřeba žádné externí zařízení. MCU Disponuje 512 KBytes paměti flash a 128 KBytes paměti SRAM. Deska obsahuje pět AD převodníků, dvanáct časovačů, tři SPI a mnoho dalších periférií. Pro projekt byla vybrána 3,3 V logika. [24][25][26]



Obr. 2.4: Nucleo STM32G474-RE[25]

IHM16M1

X-NUCLEO-IHM16M1 je výkonová deska pro ovládání elektrických motorů od STMicroelectronics. Srdcem desky je integrovaný obvod STSPIN830, nízkonapěťový a efektivní řadič s integrovanými funkcemi. Deska je vybavena kompletní sadou ochran,

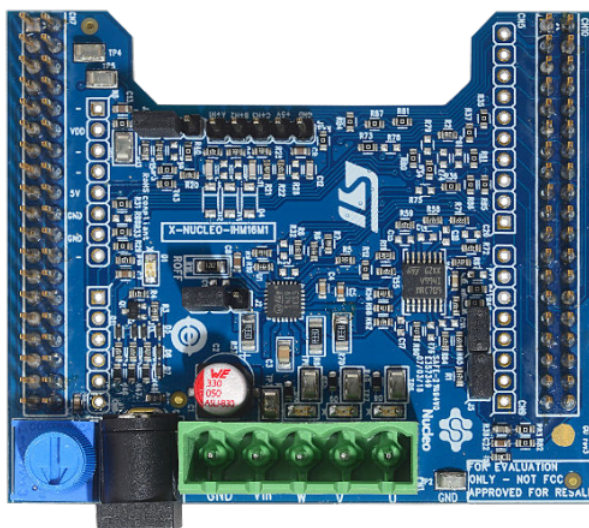
omezením výstupního proudu PWM s nastavitelným prahem a podporou pro snímání třífázového proudu pomocí jednoho (patent od ST) nebo tří bočníků. Desku je možné nasadit na Nucleo STM32G474-RE.

STSPIN830 obsahuje H-můstek pro řízení třífázových motorů. H-můstek se skládá ze tří větví, kde každá větev má horní a dolní část, takže je tvořen celkem šesti tranzistory. Pro ovládání horní větve je použit PWM signál z MCU a pro dolní větev je použita komplementární varianta PWM. Čip má zabudovanou funkci vlastního generování komplementární PWM, proto je z MCU nutné přivést pouze řídicí PWM signál.

Deska podporuje více algoritmů řízení. Pro nastavení FOC je nutné provést následující hardwarové změny. Rozpojit JP4 a JP7, na spodní straně napájet můstek, propojit J5 a J6, propojit J2 na pozici 2-3 a J3 na pozici 1-2.

Pro správnou funkci IHM16M je potřeba přivést správné signály z MCU na potřebné piny. Vstupy do desky jsou tři PWM signály (INU, INV, INW), tři signály pro povolení jednotlivých fází U, V, W (ENU, ENV, ENW) a jeden signál, který povoluje všechny tři výstupní fáze (EN/FAULT). Výstupy desky jsou tři proudy, které jsou měřeny pomocí ADC z MCU („Current feedback phase“ U, V, W). Přesné zapojení jednotlivých pinů je v tabulce 1. v kapitole Nucleo STM32 G474-RE.

Pro správné měření proudu je potřeba vzít v potaz velikost odporu bočníku. Pro přesnější měření se využívají operační zesilovače. Výkonová deska má vlastní operační zesilovače, proto je nutné při výpočtu proudu vzít v potaz i zesílení. [27][24]



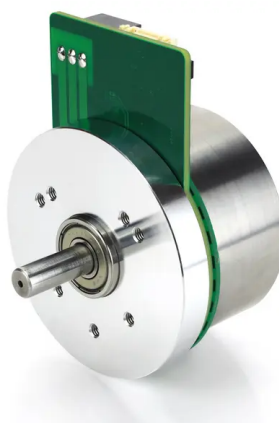
Obr. 2.5: Výkonová deska IHM16M[27]

BLDC Motor DFA68

Pro projekt byl použit BLDC motor DFA68M024035-E. Tento typ motoru je vybaven zabudovaným enkodérem pro snímání polohy motoru. Motor má 14 pólů, tedy 7 polových dvojic. Podrobnější informace o motoru jsou shrnuty v tabulce 2.5.[28]

Tab. 2.5: Technické parametry použitého motoru

Parametr	Symbol	Hodnota
Jmenovitý proud	I_{rated}	5,8 A
Špičkový proud	I_{peak}	17 A
Jmenovité napětí	U_{rated}	24 V
Jmenovitý výkon	P_{rated}	106 W
Jmenovité otáčky	n_{rated}	3500 min ⁻¹
Jmenovitý moment	M_{rated}	29 Ncm
Špičkový moment	M_{peak}	87 Ncm
Momentová konstanta	k_t	5 Ncm/A
Odpor (fáze-fáze)	R_{L-L}	0,25 Ω
Indukčnost (fáze-fáze)	L_{L-L}	0,22 mH
Moment setrvačnosti rotoru	J	1000 gcm ²
Hmotnost	m	0,5 kg
Typ enkodéru	—	inkrementální
Rozlišení enkodéru	—	4096 CPR



Obr. 2.6: BLDC motor DFA68M024035-E[28]

2.2.3 Nastavení periférií

Tato kapitola představuje přehled nastavení potřebných periférií MCU k správnému fungování FOC. Hodnoty v tabulkách odpovídají nastavení v CubeMX. Parametry, které nejsou zmíněny, zůstaly nezměněny.

GPIO

General-Purpose Input/Output zkráceně GPIO jsou univerzální piny na řídicí desce. V tabulce 2.6 je přehled použitých pinů pro řízení PWM, snímání proudů pomocí ADC a získání úhlu otáček motoru pomocí časovače.

Tab. 2.6: Použité piny a jejich význam

Periferie	Pin	Funkce	Význam
TIM1	PA8	TIM1_CH1	INU - PWM signál pro fázi U
	PA9	TIM1_CH2	INV - PWM signál pro fázi V
	PA10	TIM1_CH3	INW - PWM signál pro fázi W
ADC1	PA0	ADC1_IN2	Měření proudu fází U
	PC1	ADC1_IN14	Měření proudu fází V
	PC0	ADC1_IN15	Měření proudu fází W
TIM8	PC6	TIM8_CH1	Enkodérový kanál A
	PC7	TIM8_CH2	Enkodérový kanál B
GPIO	PA11	GPIO_OUT	EN/F - Povolení všech fází
	PC10	GPIO_OUT	ENU - Povolení fáze U
	PC11	GPIO_OUT	ENV - Povolení fáze V
	PC12	GPIO_OUT	ENW - Povolení fáze W
	PC13	GPIO_IN	Modré tlačítko na desce
LPUART1	PA3	LPUART1_RX	Komunikace s prostředím SIMULINK
	PA2	LPUART1_TX	Komunikace s prostředím SIMULINK

Timer

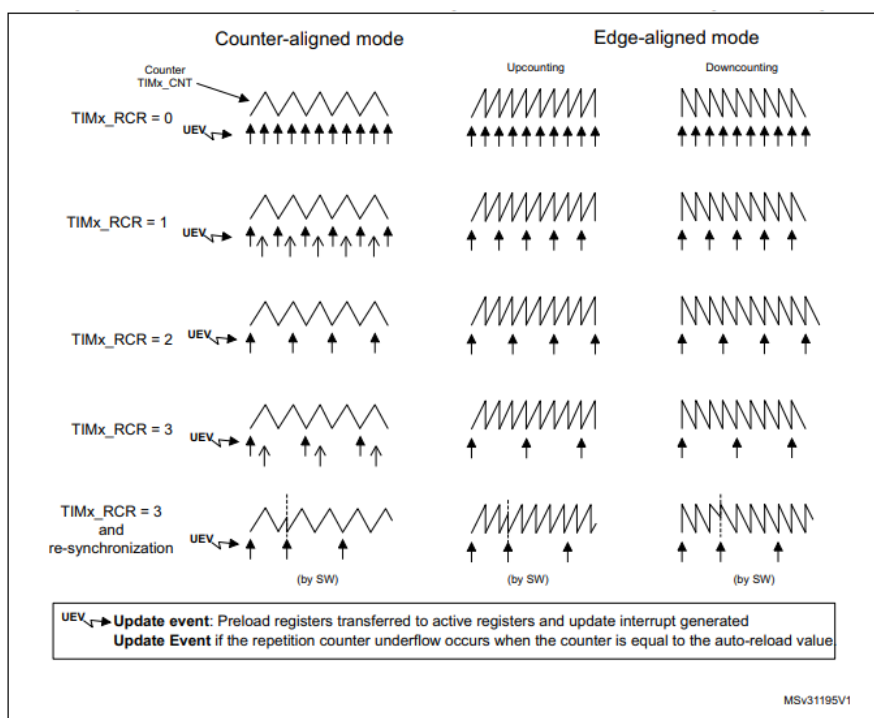
Timer, neboli časovač, je základní periferie u mikrokontrolérů, která slouží pro časování a generování časových intervalů. Dokáže časovat procesy, události, vyvolávat přerušení nebo vytvářet PWM.

STM32G474-RE disponuje více časovači, prvním je „general-purpose timer“, který má funkce jako „one-pulse mode“ a „input capture“. Dalším typem jsou „advanced timers“ (například TIM1), tyto časovače mají větší škálu možností, například možnost generování komplementárních kanálů s vložením mrtvé doby pro ovládání motorů nebo TRGO pro aktivaci ADC v požadovaný moment.

Časovač se inkrementuje každým tikem hodinového signálu mikroprocesoru, dokud nedosáhne svého stropu. Tím dojde k přetečení, a je vyvoláno přerušení a časovač začíná znovu. Systémový hodinový signál je možné podělit „prescalerem a tím zpomalit přetečení.

$$\text{strop časovače} = \frac{SYSCLK}{(PSC + 1)(ARR + 1)(RCR + 1)} \quad (2.3)$$

Timer1 na STM32G474-RE umožňuje použití a nastavení PWM, jako jsou módy 1 a 2. Mód 2 slouží pro generování komplementární PWM. Časovač má tři režimy: „Upcounting mode“ pro postupné počítání od 0 do stropu (registr ARR). „Downcounting mode“ počítá od stropu do 0 a „Center-aligned mode“ počítá od 0 do stropu a znovu do 0. Při „Center-aligned mode 3“ je vlajka přetečení vyvěšena vždy, když časovač dosáhne stropu nebo nuly. Vliv na generování události přetečení má i „Repetition counter“(RCR). Pokud byla do RCR zapsána hodnota 1 a k zápisu došlo po spuštění časovače, vyvěsí se vlajka při přetečení. Pokud k zápisu hodnoty došlo před spuštěním časovače, vlajka bude vyvěšena při podtečení.[29]



Obr. 2.7: Módy čítače[29]

Tab. 2.7: Shrnutí nastavení časovače TIM1

Parametr	Hodnota
Channel 1	PWM Generation CH1
Channel 2	PWM Generation CH2
Channel 3	PWM Generation CH3
Counter Mode	Center Aligned mode 3
Counter Period (ARR)	4250
Repetition Counter (RCR)	1
Trigger Event Selection TRGO	Update Event

Tab. 2.8: Nastavení TIM1 PWM Generation Channels

Kanál	Parametr	Hodnota
1	Mode Pulse	PWM Mode 2 2125
2	Mode Pulse	PWM Mode 2 2125
3	Mode Pulse	PWM Mode 2 2125

Při implementaci měření proudů pomocí ADC v Simulinku bylo zjištěno, že pro správné načasování měření je potřeba nastavit parametry uvedené v tabulce 2.9. Tedy toto nastavení platí pouze pro aplikaci v simulinku a nevztahuje se na implementaci v C.

Tab. 2.9: Nastavení TIM1 PWM Generation Channels pro Simulink

Kanál	Parametr	Hodnota
1	Mode	PWM Mode 1
2	Mode	PWM Mode 1
3	Mode	PWM Mode 1

Pro měření úhlu natočení motoru je využit zabudovaný inkrementální enkodér. Snímač pracuje na principu generování dvou fázově posunutých obdélníkových signálů (kvadrurní signál) na kanálech A a B. Fázový posun signálů o 90° umožňuje časovači na STM32 určit směr otáčení motoru.

Timer 8 je konfigurován v režimu enkodéru (Encoder Mode), který automaticky inkrementuje nebo dekrementuje hodnotu čítače na základě hran vstupních signálů. Nastavení TIM8 je v tabulce 2.10.

Tab. 2.10: Shrnutí nastavení časovače TIM8

Parametr	Hodnota
Combined Channels	Encoder Mode
Counter Period (ARR)	16383
Encoder Mode	Encoder Mode TI1 and TI2
Input filtr (ARR)	3
Input filtr (ARR)	3

ADC

„Analog digital converter“, česky analogově digitální převodník, je poměrně komplikovaná periferie, která je součástí téměř každého mikrokontroléru. Stará se o převod analogového signálu na signál diskretní. Převod je nezbytný pro moderní aplikace, protože mikroprocesory nejsou uzpůsobeny pro práci s analogovými veličinami.

STM32G474-RE obsahuje pět AD převodníků, kde každý má několik kanálů. Kanály mají možnost nastavení „Single ended“, které měří relativně vůči společnému GND, a „Differential“, které měří rozdíl mezi dvěma kanály.

Periferie nabízí mnoho možností nastavení, vypsány jsou pouze ty důležité pro tuto práci. „Regular conversion mode“, tento režim slouží pro jednoduchá měření, kde jsou kanály měřeny postupně podle daného pořadí. „Injected conversion mode“ je určen pro speciální měření, kde je kladen důraz na měření v určitém čase, často spouštěn externím signálem. Má kvůli náročnějšímu převodu menší počet dostupných kanálů ve srovnání s regular módem. „Single conversion mode“ provede jednu konverzi každého kanálu a čeká na další pokyn k další konverzi. „Continuous conversion mode“ provede konverzi jednotlivých kanálů postupně a při posledním kanálu se restartuje a začíná od začátku, je aplikovatelný pouze pro regular conversion. „Discontinuous mode“, v tomto režimu jsou kanály v sekvenci měřeny postupně, přičemž po každém převodu čeká ADC na nový spouštěcí signál pro další měření. Jednotlivé módy mohou být spuštěny programem nebo pomocí externího „trigger“ signálu TRGO.[29][30]

Tab. 2.11: Přehled nastavení parametrů ADC1

Parametr	Hodnota
IN1	IN1 Single-ended
IN6	IN6 Single-ended
IN7	IN7 Single-ended
Scan Conversion mode	Enabled
End Of Conversion Selection	End Of sequence of conversion
Enabled Regular Conversions	Disable
Enabled Injected Conversions	Enabled
Number Of Conversion	3
External Trigger Source	Timer 1 Trigger Out event
Rank 1	Channel 1
Rank 2	Channel 7
Rank 3	Channel 6

2.2.4 Implementace algoritmu v jazyce C

Nastavení periferií bylo provedeno v CubeMX. Do vytvořeného projektu bylo implementováno vektorové řízení. Celá aplikace běží v main.c. Nejdříve bylo potřeba povolit jednotlivé fáze desky, odstartovat běh časovačů a spustit ADC v režimu přerušování. Pro většinu matematických výpočtů byla použita knihovna math.h. Diskrétní PI regulátor byl realizován na základě rovnic z [31], implementace je v příloze A.4.

Program je implementován pomocí stavového automatu. Prvním stavem je ADC _ CALIBRATION, zde je určen Offset měření pro každou fázi. Další stav ALIGN motor zarovná a uloží offset úhlu, o který se posunul. Následně program přejde do stavu ZERO_PHASE, kde je napětí na fázích vynulováno. Posledním stavem je RUN. Kvůli požadavku na časování je logika implementována v obsluze přerušování (Callback funkce) ADC převodníku.

V Callback funkci jsou nejprve vyčteny hodnoty ADC, které jsou následně přepočteny na napětí pomocí vzorce 2.4 a dále přepočteny na proud přes vzorec 2.5.

$$U_{fáze} = ADC_{raw} \cdot \frac{U_{ref}}{2^n - 1} \quad (2.4)$$

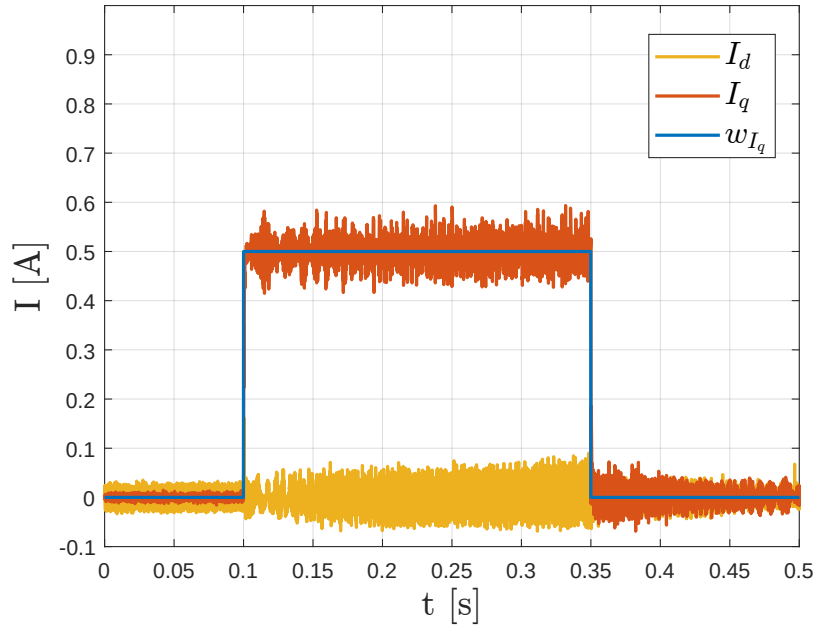
$$I_{fáze} = \frac{U_{fáze} - Offset}{Zesílení_{OPAMP} \cdot Odpor_{Shunt}} \quad (2.5)$$

Úhel je potřeba dopočítat z hodnoty časovače 8. Získaný úhel je ale mechanický a je potřeba přepočítat ho na elektrický do rozsahu $\pm\pi$ pomocí vzorce 2.6.

$$\theta_{el} = p \cdot (\theta_{mech} - \theta_{offset}) \quad (2.6)$$

Elektrický úhel se používá v transformacích pro Přechod do D-Q souřadnicového systému a zpět. Regulační smyčka je implementována stejně jako v kapitole 2.1.3 Vektorové řízení. Přehled kódu je v příloze A.

Jak už bylo zmíněno, parametry regulátorů byly navrženy experimentálně a jsou vypsány v Tabulce 2.3 a 2.4. Průběh proudů I_q a I_d při točící se hřídeli byl vynesena do grafu na obrázku 2.8. Přejímová charakteristika proudu I_d je zaznamenána na obrázku 2.10.



Obr. 2.8: Průběh Proudů při běžícím motoru implementace v jazyce C

Z průběhu proudu i_q je patrné, že regulátor dosáhne žádané hodnoty s minimálním překmitem a doba regulace je okolo 2 ms. Zvlnění Proudů je zapříčiněno šumem ADC převodníků, spínací frekvencí PWM a nepřesným měřením polohy rotoru.

Složka proudu i_d je regulována na nulovou hodnotu, aby bylo dosaženo maximálního momentu. Při prudké změně proudu může dojít ke křížové vazbě mezi D a Q složkou. Regulátor ale úspěšně chybu eliminuje.

Lze pozorovat, že chyba složek se postupem času zvětšuje. Při konstantním žádaném momentu bez připojené zátěže dochází k trvalému nárůstu otáček motoru;

to je zapříčiněno samotnou řídicí smyčkou. Momentová regulace bez zátěže neustále zvyšuje otáčky motoru, až se systém stane nestabilním.

2.2.5 Implementace algoritmu v Simulinku

Nastavení Simulinku

Pro správné generování kódu a zajištění kompatibility se zvoleným mikrokontrolérem je nutné v prostředí Simulink nakonfigurovat parametry v sekci Hardware Implementation. V záložce Connectivity je nutné nastavit komunikační rozhraní (LPUART1) a port, ke kterému je MCU připojeno. Dále v záložce ADC1 je nezbytné povolit možnost Enable Injected EoCS Interrupt.

Frekvence měření proudu je dána signálem TRGO, tato frekvence musí být zadána i do sekce Solver pro správný běh simulace. Společně s nastavením frekvence je potřeba nastavit Type na Fixed-step a Solver na Discrete(no continuous states)

Nastavení funkčních STM32 bloků

Programování platformy STM32 v Simulinku vyžaduje nastavení periférií v CubeMX, ale také specifikaci nastavení v samotných funkčních blocích programu. V tabulce 2.12 je uveden přehled potřebných nastavení pro bloky z knihovny STM32 a bloku Function-Call Subsystem.

Tab. 2.12: Nastavení programových bloků

Název bloku	Parametr	Hodnota
Encoder	Timer module	TIM8
	Sample time	50 μs
Hardware Interrupt	Interrupt group	Analog to digital converter
	Interrupt name	ADC1_2_IRQHandler
	Select event	ADC1 Inj EoCS
Analog to Digital Converter	ADC module	ADC1
	Conversion group	Injected
	Trigger mode	Read results only
	Number of conversions	3
	Sample time	-1
PWM Output	Timer module	TIM1
	Duty cycle unit	Counts
	Enable channel	1,2,3
	Set the repetition counter	Enable
Call-back function	Trigger type	Function-call
	States when enabling	Held
	Sample time type	Triggered

Realizace řídicího algoritmu

Algoritmus v Simulinku je ekvivalentem programu v jazyku C. Jedinou změnou je způsob přechodu mezi stavy zarovnání a běhu motoru. Přechod byl vyřešen pomocí tlačítka zabudovaného na vývojové desce. Celá řídicí smyčka byla umístěna do subsystému vyvolaného přerušením od AD převodníku. Do subsystému vstupuje pouze měřený úhel a stisk tlačítka.

Přerušení je asynchronní událost, proto bylo nutné implementovat bloky Rate transition; ty zajišťují bezpečnou výměnu dat mezi částmi s jinou periodou vzorkování. Dále bylo provedeno přetypování na single, aby se zamezilo vysoké výpočetní náročnosti spojené s typem double.

Na obrázku 2.9 je znázorněno hlavní zapojení algoritmu, které zahrnuje vyčítání úhlu rotoru, přechod mezi jednotlivými stavy pomocí tlačítka, povolení jednotlivých fází a spouštění řídicí smyčky prostřednictvím přerušení.

V příloze A.6 je uvedena struktura subsystému vyvolaného přerušením, tedy proudová (momentová) smyčka FOC. Jednotlivé parametry pro výpočty byly převzaty z implementace v jazyce C.

Na obrázku 2.10 je znázorněn průběh proudů v dq souřadnicovém systému mo-

2.3 Porovnání modelu a reality

Pro porovnání kvality regulace mezi jednotlivými implementacemi je použit obrázek 2.11. Graf zachycuje jednotlivé odezvy systému na jednotkový skok.

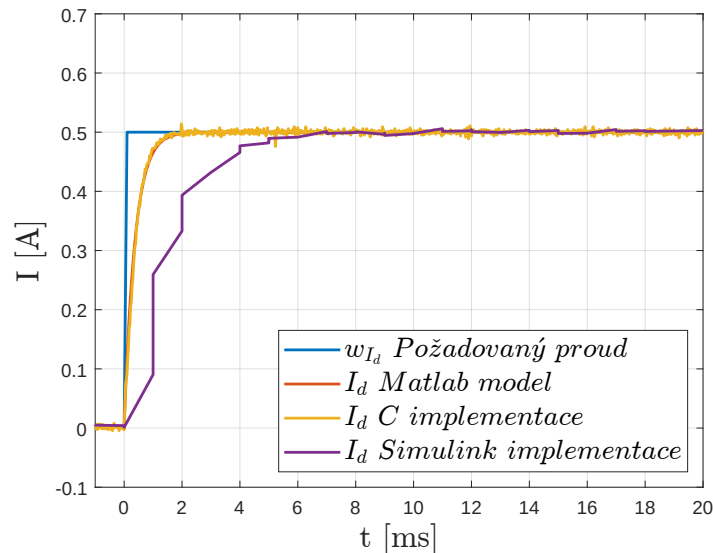
Z grafu lze pozorovat, že přechodová charakteristika implementace v jazyku C je bez překmitu, čímž je zamezeno překročení požadované hodnoty proudu. Přechodový děj trvá přibližně 2 ms, to je dostatečně rychlá odezva pro vektorové řízení BLDC motoru. Obrázek 2.8 potvrzuje, že regulace je dostatečně rychlá pro obě složky, i při roztočené hřídeli.

Matlabový model byl odvozen z přechodové charakteristiky proudu i_d reálného systému. Z toho důvodu se téměř shoduje s průběhem implementace v jazyku C, tedy je jeho idealizovanou verzí bez šumu.

Implementace v Simulinku byla vytvořena jako ekvivalent k aplikaci v jazyku C. Přechodová charakteristika je opět bez překmitu. I přes stejné nastavení parametrů lze z grafu určit, že doba ustálení trvá přibližně 7 ms, tedy je pomalejší než C implementace. Najít příčinu rozdílu však nebylo součástí zadání práce, a proto zde není uvedena.

Rozdílné vzorkování přechodových charakteristik je zapříčiněno způsobem, jakým byly pořízeny. Oba průběhy byly naměřeny pomocí CubeMonitoru. Rozdílem je, že implementace pomocí CubeIDE dovoluje použití snap-shot módu, kdežto simulink dovoluje použití pouze direct módu.

Na základě uvedených výsledků lze konstatovat, že vektorové řízení BLDC motoru bylo úspěšně implementováno ve všech navržených variantách.



Obr. 2.11: Přehled průběhů proudů I_d

3 Návrh a realizace řízení systému Furuta pendulum

Tato kapitola se zabývá návrhem řízení rotačního inverzního kyvadla. Nejprve byl navržen a sestaven samotný přípravek, poté byl vytvořen nelineární matematický model v prostředí Simulink vycházející z parametrů sestaveného kyvadla. Na modelu byl navržen stavový zpětný regulátor a Luenbergerův estimátor. Navržený regulátor a estimátor byly následně použity pro řízení reálné soustavy.

3.1 Realizace rotačního inverzního kyvadla

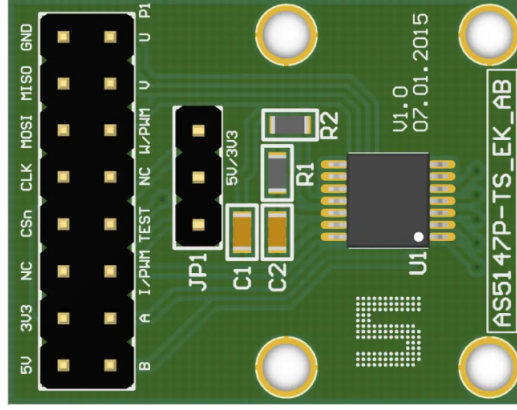
Inverzní kyvadlo bylo realizováno pomocí součástek vytisknutých na 3d tiskárně. V programu SOLIDWORKS bylo navrženo rameno, které je přichyceno přes navrženou spojku k hřídeli motoru. Dále bylo vytvořeno samotné kyvadlo se závitem na konci pro možnost volby různých hmotností. Kyvadlo je připojeno přes železnou tyč, která je vedena skrz dvě ložiska pro stabilitu, a na konci tyče je umístěn navržený držák pro magnet. Na rameno byl vytvořen podstavec pro magnetický enkodér AS5147P, který snímá polohu magnetu a odesílá informaci o úhlu kyvadla(θ) do mikrokontroléru. Díly jsou v příloze C a finální realizace Furuta pendulum lze najít v příloze B.3.

AS5147P

Poslední přidanou periferií do systému je magnetický rotační senzor pro měření úhlu v celém rozsahu 360°. Čip má volatilní registry pro přístup k hodnotě měřeného úhlu. A nevolatilní registry, jednorázově programovatelnou paměť pro speciální případy. Desku je možné napájet napětím 5V i 3,3V. Pro napájení 3,3V je potřeba přepájet rezistor z pozice R1 na pozici R2. Informace o úhlu je odesílána přes SPI. Čip funguje pouze v módu slave a maximální komunikační rychlost je až 10 MHz. [32]

3.2 Dynamický model soustavy

Základní pohybové rovnice a z nich odvozené nelineární rovnice byly převzaty z [17]. Nelineární model byl využit v simulinku jako simulace reálné soustavy a linearizovaný model byl použit pro návrh regulátorů a estimátoru.



Obr. 3.1: Magnetický senzor AS5147P[17]

3.2.1 Matematický popis

Diferenciální rovnice

Pohybové rovnice v integračním tvaru (24) prezentované v článku však uvažují ideální systém bez tlumení. Pro simulaci a řízení reálného systému bylo proto nezbytné tyto rovnice rozšířit o členy viskózního tření.

$$\frac{d}{dt}\phi = \dot{\phi} \quad (3.1)$$

$$\begin{aligned} \frac{d}{dt}\dot{\phi} = \frac{1}{\alpha\beta - \gamma^2 + (\beta^2 + \gamma^2)\sin^2\theta} \bigg\{ & \beta\gamma(\sin^2\theta - 1)\sin\theta\dot{\phi}^2 - 2\beta^2\cos\theta\sin\theta\dot{\phi}\dot{\theta} + \beta\gamma\sin\theta\dot{\theta}^2 \\ & - \gamma\delta\cos\theta\sin\theta + \beta(\tau - b_a\dot{\phi}) + \gamma\cos\theta(b_p\dot{\theta}) \bigg\} \end{aligned} \quad (3.2)$$

$$\frac{d}{dt}\theta = \dot{\theta} \quad (3.3)$$

$$\begin{aligned} \frac{d}{dt}\dot{\theta} = \frac{1}{\alpha\beta - \gamma^2 + (\beta^2 + \gamma^2)\sin^2\theta} \bigg\{ & \beta(\alpha + \beta\sin^2\theta)\cos\theta\sin\theta\dot{\phi}^2 + 2\beta\gamma(1 - \sin^2\theta)\sin\theta\dot{\phi}\dot{\theta} \\ & - \gamma^2\cos\theta\sin\theta\dot{\theta}^2 + \delta(\alpha + \beta\sin^2\theta)\sin\theta \\ & - \gamma\cos\theta\tau + \gamma\cos\theta(b_a\dot{\phi}) - (\alpha + \beta\sin^2\theta)b_p\dot{\theta} \bigg\} \end{aligned} \quad (3.4)$$

Z diferenciálních rovnic vyplývá, že furuta pendulum má čtyři stavové veličiny odpovídající úhlům a úhlovým rychlostem ramene a kyvadla. Úhel ϕ reprezentuje natočení hřídele motoru, a tedy úhel natočení ramene. Derivací úhlu je $\dot{\phi}$, což je

úhlová rychlost ramene. Analogicky θ představuje úhel kyvadla a $\dot{\theta}$ jeho úhlovou rychlost.

Lineární náhrada

Protože návrhové metody regulátorů pracují s linearizovanými rovnicemi, je nutné převést nelineární rovnice na jejich lineární náhrady. K převodu se nejčastěji používá metoda rozvoje do Taylorovy řady. Ta se obvykle provádí okolo rovnovážného stavu, u inverzního kyvadla tomu odpovídá vzpřímená poloha kyvadla. Tento stav je definován jako:

$$x_0 = [\phi_0 \quad \dot{\phi}_0 \quad \theta_0 \quad \dot{\theta}_0]^T = [0 \quad 0 \quad 0 \quad 0]^T, \quad \tau_0 = 0 \quad (3.5)$$

Aplikací Taylorova rozvoje prvního řádu (Jacobiho matice) na nelineární systém rovnic získáme lineární stavový model ve tvaru $\dot{x} = Ax + B\tau$. Pro výpočet lze použít matlabovou funkci za předpokladu, že máme definované nelineární rovnice. Výsledné matice A a B se započtením viskózního tření mají tvar:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{\beta b_a}{\alpha\beta - \gamma^2} & -\frac{\gamma\delta}{\alpha\beta - \gamma^2} & \frac{\gamma b_p}{\alpha\beta - \gamma^2} \\ 0 & 0 & 0 & 1 \\ 0 & \frac{\gamma b_a}{\alpha\beta - \gamma^2} & \frac{\alpha\delta}{\alpha\beta - \gamma^2} & -\frac{\alpha b_p}{\alpha\beta - \gamma^2} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{\beta}{\alpha\beta - \gamma^2} \\ 0 \\ -\frac{\gamma}{\alpha\beta - \gamma^2} \end{bmatrix} \quad (3.6)$$

Z rovnice 3.6 lze pozorovat, že matice B má pouze jeden sloupec, oproti článku, kde autor pracuje se dvěma sloupci, tedy se dvěma vstupy. Sestrojený přípravek má ale pouze jeden motor připojený k rameni, ovšem hřídel kyvadla se otáčí volně a není k ní připojen aktuátor. Pro lineární stavový model reálné soustavy je proto využit pouze první sloupec matice B , který reprezentuje jediný akční zásah. Tedy se jedná o poddimenzovanou soustavu.

Odvozené nelineární pohybové rovnice byly implementovány do matlabové funkce. Vstupem tohoto modelu je budící moment motoru a aktuální stavový vektor soustavy. Výstupem funkce je vektor derivací stavových veličin, který je dále integrován.

Pro úplný popis linearizovaného modelu v okolí horní rovnovážné polohy je dále nutné definovat matice C a D , které určují vztah mezi stavovými veličinami, vstupy a výstupy systému: $y = Cx + Du$.

Vzhledem k tomu, že systém je vybaven pouze dvěma snímači polohy, a to pro snímání natočení hřídele a úhlu kyvadla, jsou přímo měřenými veličinami úhly ϕ a θ . Úhlové rychlosti nejsou měřeny přímo, ale jsou dále dopočítány. Z toho vyplývá, že pozorovací matice C a matice přímých vazeb budou v následujícím tvaru:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad D = [0] \quad (3.7)$$

3.2.2 Identifikace parametrů

Parametry reálné soustavy byly změřeny a dopočítány, dále byla provedena simulace s realitou a hodnoty byly doladěny experimentálně.

Symbol	Název parametru	Hodnota	Jednotka
J	Moment setrvačnosti motoru	0,0009	$\text{kg} \cdot \text{m}^2$
M	Hmotnost závaží na konci kyvadla	0,023	kg
m_a	Hmotnost ramene	0,072	kg
l_a	Délka ramene	0,16	m
m_p	Hmotnost kyvadla	0,010	kg
l_p	Celková délka kyvadla	0,132	m
g	Tíhové zrychlení	9,81	$\text{m} \cdot \text{s}^{-2}$
b_ϕ	Koeficient viskózního tření ramena	0,013	$\text{N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}$
b_p	Koeficient viskózního tření kyvadla	0,00022	$\text{N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}$

Pro určení momentu setrvačnosti byla využita Steinerova věta, obecný vztah pro přepočet momentu setrvačnosti I k ose, která prochází těžištěm tělesa, a má tvar:

$$I = I_T + m \cdot d^2 \quad (3.8)$$

Osa otáčení byla posunuta ze středu na kraj pomocí $d = \frac{l}{2}$. Výsledný výraz tedy slouží k výpočtu momentu setrvačnosti na konci tyče.

$$J = \frac{1}{3} m_a l_a^2 = \frac{1}{3} \cdot 0,072 \cdot 0,16^2 \approx 6,14 \cdot 10^{-4} \text{ kg} \cdot \text{m}^2 \quad (3.9)$$

Na základě porovnání průběhů úhlu nelineárního modelu v Matlabu s naměřenými daty z reálného laboratorního přípravku byla iterativním laděním stanovena finální hodnota J . Podobným způsobem byl získán moment setrvačnosti kyvadla $J_p = 0,000458 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}$ který byl použit pro další výpočty.

Pro určení koeficientů viskózního tření byl změřen průběh úhlu kyvadla při volném kmitání. Následně byla využita metoda logaritmického dekrementu, která popisuje rychlost poklesu amplitudy tlumeného kmitání. Z naměřeného průběhu byly odečteny hodnoty amplitud dvou maxim a jejich časy.

Nejprve byla určena perioda kmitů a úhlová frekvence:

$$T_d = t_2 - t_1 = 2,4 - 1,6 = \mathbf{0,8 \text{ s}} \quad (3.10)$$

$$\omega_d = \frac{2\pi}{T_d} = \frac{2\pi}{0,8} = \mathbf{7,854 \text{ rad/s}} \quad (3.11)$$

Tab. 3.1: Odečtené hodnoty kmitů kyvadla pro identifikaci tření

Špička	Čas [s]	Amplituda [rad]
1. maximum	1,6	2,0
2. maximum	2,4	1,56

Dále byl určen logaritmický dekrement, činitel tlumení a vlastní frekvence:

$$\delta = \ln \left(\frac{X_1}{X_2} \right) = \ln \left(\frac{2,0}{1,56} \right) = \ln(1,282) = \mathbf{0,248} \quad (3.12)$$

$$\zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}} = \frac{0,248}{\sqrt{39,478 + 0,061}} = \frac{0,248}{6,288} = \mathbf{0,0394} \quad (3.13)$$

$$\omega_0 = \frac{\omega_d}{\sqrt{1 - \zeta^2}} = \frac{7,854}{\sqrt{1 - 0,0394^2}} \approx \mathbf{7,860} \text{ rad/s} \quad (3.14)$$

Pro výpočet koeficientu viskózního tření je dále nutná znalost momentu setrvačnosti kyvadla, který byl určen v předchozí části.

$$b_p = 2 \cdot \zeta \cdot \omega_0 \cdot J_p = 0.000284 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1} \quad (3.15)$$

Obdobně bylo získáno viskózní tření ramene; hodnoty získané výpočtem byly následně doladěny experimentálně na základě porovnání simulace s reálnou soustavou.

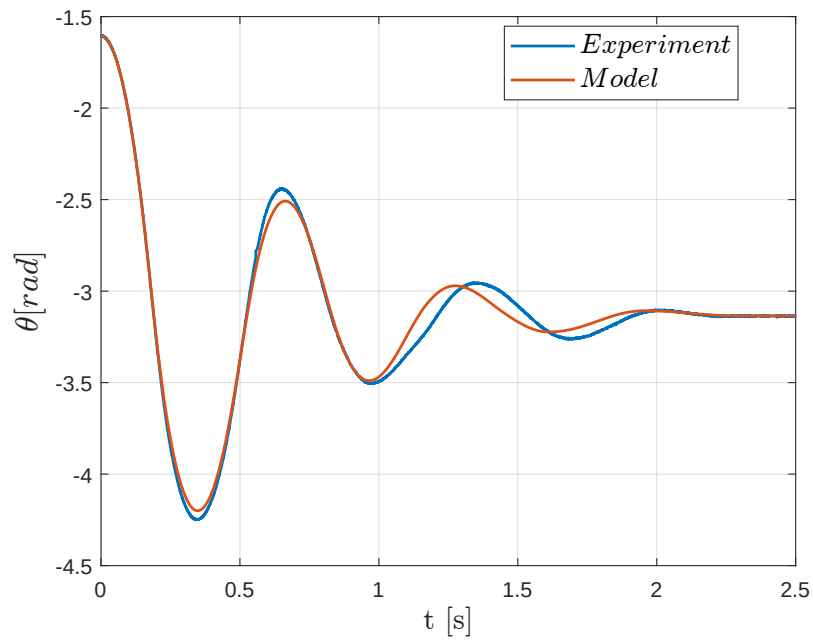
Obrázek 3.1 ukazuje naměřený průběh úhlu ramene a podle něj nastavený model rotačního inverzního kyvadla podle rovnic a parametrů zmíněných výše. Obdobně obrázek 3.2 zobrazuje naměřený a simulovaný úhel kyvadla.

3.3 Řídící algoritmus

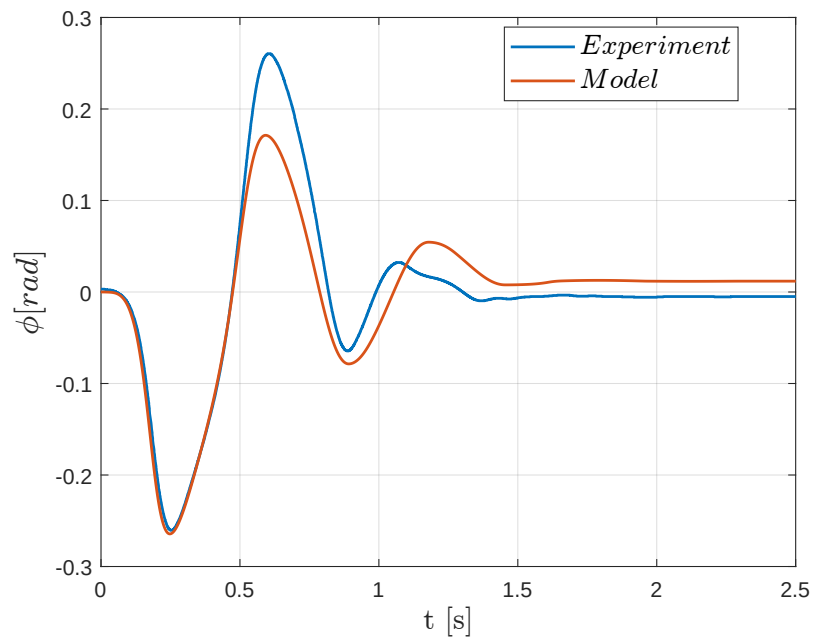
3.3.1 Zpětná stavová regulace

Jednou z nejznámějších metod řízení je PID regulátor, který umožňuje potlačení regulační odchylky a vyregulování veličiny na požadovanou hodnotu. Vzhledem k tomu, že integrační složka je již obsažena v PI regulátorech momentové smyčky FOC, není použití další integrační složky v nadřazeném regulátoru vhodné. Pro lze kyvadlo stabilizovat i pomocí PD regulátoru, přičemž podobné realizace lze nalézt i v odborné literatuře.

Pro řízení rotačního inverzního kyvadla však bývá vhodnější využití metod založených na stavovém popisu systému. Pokud je znám matematický model soustavy



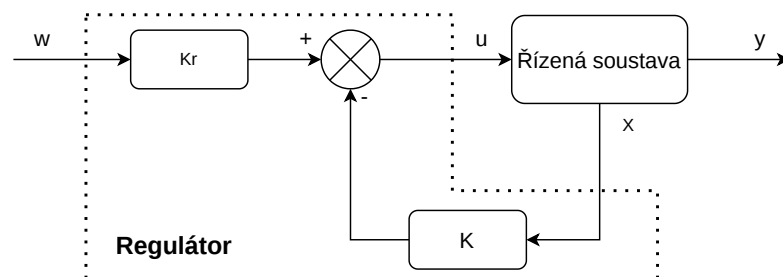
Obr. 3.2: Srovnání úhlu kyvadla modelu s realitou



Obr. 3.3: Srovnání úhlu ramene modelu s realitou

a všechny její hlavní stavy, lze použít zpětnovazební stavové řízení (state-feedback control). Tato metoda umožňuje současné řízení všech stavových veličin systému a zpravidla poskytuje kvalitnější dynamické vlastnosti než klasické PID regulátory

Stavové řízení je graficky znázorněno na obrázku 3.3. Jednotlivé veličiny jsou



Obr. 3.4: Blokové schéma stavové zpětné vazby

definovány následovně:

- w – požadovaná veličina (poloha ramene)
- u – akční zásah (moment)
- x – stavový vektor systému $([\phi, \dot{\phi}, \theta, \dot{\theta}])$
- y – výstup ze soustavy
- K – matice zesílení stavové zpětné vazby
- K_r – škálovací člen, zajištění nulové regulační odchylky v ustáleném stavu.

Princip zpětnovazebního stavového řízení se liší od klasických metod regulace tím, že zpětná vazba není tvořena pouze výstupní veličinou systému a regulační odchylkou. Zpětná stavová vazba, jak už název napovídá, má ve zpětné vazbě přímo všechny stavové veličiny systému. Díky zesílení K lze změnit chování vlastních čísel matice A a tedy změnit polohu pólů systému. Například, jako v případě této práce, kdy vlastní čísla matice A jsou:

$$\lambda(\mathbf{A}) = \{0; -9.0863; -2.9403; 3.1782\} \quad (3.16)$$

Systém je tedy nestabilní, protože má jeden pól v pravé polorovině komplexní roviny. Avšak pomocí matice K lze najít takový koeficient, který kladný pól přesune do levé poloroviny, a tím se stane stabilním.

V prostředí MATLAB bylo ověřeno, že systém je plně říditelný, což znamená, že jej lze pomocí vhodného akčního zásahu převést z jakéhokoli počátečního stavu do požadovaného stavu.

Pokud je systém říditelný, lze přistoupit k návrhu matice K . Základní a názornou metodou pro návrh zpětného stavového regulátoru je metoda rozmístění pólů (Pole Placement).

3.3.2 Metoda rozmístění pólů

Princip této metody spočívá v tom, že dynamické chování uzavřeného regulačního obvodu je určeno polohou jeho pólů v komplexní rovině. Lineární stavový zákon $\dot{u} = -Kx$ vyjadřuje, že stavy systému jsou vynásobeny maticí A a znaménko mínus reprezentuje zápornou zpětnou vazbu. Pokud dosadíme lineární stavový zákon do popisu soustavy, získáme matici A uzavřeného regulačního obvodu $A_{cl} = (A - BK)$.

$$\dot{x} = Ax + B(w \cdot K_r - Kx) \quad (3.17)$$

$$\dot{x} = (A - BK)x + B \cdot K_r \quad (3.18)$$

Cílem metody je nalézt takovou matici K , aby charakteristická rovnice uzavřeného systému byla tvaru:

$$\det(A_{cl} - \lambda I) = 0 \quad (3.19)$$

Prvky matice K lze získat ručním roznásobením a porovnáním koeficientů výsledných polynomů. U systému čtvrtého řádu, což je rotační inverzní kyvadlo, je takový postup analyticky náročný kvůli výpočtu determinantu. Z tohoto důvodu byla pro syntézu regulátoru využita funkce `place` v prostředí MATLAB:

$$K = \text{place}(A, B, P)$$

Tato funkce, na základě matic systému A, B a uživatelem definovaného vektoru požadovaných pólů P , ze vztahu 3.19 vypočítá a vrátí vektor zesílení $K = [k_1, k_2, k_3, k_4]$, který zajistí stabilizaci kyvadla v požadované poloze.[33][34]

Iterativním postupem byly zvoleny póly uzavřeného systému, kterým odpovídá následující zesilovací matice K :

$$P = \{-1.2; -0.4; -21; -22\}$$

$$K = \{-0.0340; -0.1309; -0.7476; -0.1459\}$$

Po aplikaci zpětné stavové vazby $A_{cl} = (A - BK)$. Mají póly uzavřeného systému hodnoty, které byly vybrány. Na první pohled lze vidět, že systém má pouze záporné póly a tedy je stabilní.

$$P = \{-22.0; -21.0000; -1.2000; -0.4000\}$$

3.3.3 Lineární kvadratický regulátor

Návrhová metoda se zkráceně označuje jako LQR ("Linear Quadratic Regulator"). Předchozí metoda řízení se zabývala správným umístěním pólů uzavřené smyčky.

Pro systémy nižšího řádu to může být poměrně intuitivní, ale u složitějších systémů. může být výběr správných pólů náročný. Tento problém právě řeší návrhová metoda LQR.

Schéma zapojení zůstává stejné jako na obrázku 3.3, co se mění, je však návrh matice K. K návrhu se přistupuje jako k hledání optimálního řízení. Toho je dosaženo pomocí rovnice:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt \quad (3.20)$$

J představuje účelovou funkci, která hodnotí kvalitu regulace v čase. První člen $x^T Q x$ penalizuje velikost stavových veličin, zatímco člen $u^T R u$ penalizuje velikost akčního zásahu regulátoru. Cílem regulace je minimalizovat hodnotu J, tedy nalézt kompromis mezi rychlostí regulace a energetickou náročností řízení.

Kvadratický tvar kritéria zajišťuje, že jednotlivé členy jsou vždy kladné. Při použití lineárního kritéria by mohlo docházet ke vzájemnému odečítání kladných a záporných hodnot pod integrálem, což by neodpovídalo skutečné velikosti regulační odchylky ani akčnímu zásahu.

Cílem LQR regulátoru je nalézt matici zesílení K, která minimalizuje hodnotu funkcionálu J. Řešení této optimalizační úlohy lze získat pomocí algebraické Riccatiho rovnice:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0 \quad (3.21)$$

Výsledkem rovnice je matice P, kterou lze použít pro výpočet zesílení zpětné stavové vazby.

$$K = R^{-1} B^T P \quad (3.22)$$

$$u = -K x \quad (3.23)$$

I přes složitější matematický základ se v podstatě návrh zjednodušuje na volbu matic Q a R. Zjednodušeně řečeno, matice Q říká systému, které stavy jsou důležité pro řízení, a velikost R omezuje velikost akčního zásahu. Úloha nalezení vhodných pólů systému se tedy zjednodušila na penalizaci stavů a akčního zásahu, což je pro návrh řídicího systému intuitivnější. Návrh se ještě zjednoduší při použití Matlabové funkce `lqr`. [35][36]

$$K = \text{lqr}(A, B, Q, R)$$

Matice Q a R byly vybrány následovně:

$$Q = \begin{bmatrix} 5 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 350 & 0 \\ 0 & 0 & 0 & 0.1 \end{bmatrix}, \quad R = [10] \quad (3.24)$$

Tomuto nastavení odpovídá matice zesílení K a póly uzavřeného systému:

$$K = \{-0.7071; -1.1373; -7.1954; -1.0492\}$$

$$P = \{-86.1905; -55.7307; -0.7281 + 0.6568i; -0.7281 - 0.6568i\}$$

Z pólů systému lze usoudit, že systém je stabilní, protože póly jsou záporné. Komplexně sdružená dvojice vyjadřuje, že výsledný systém bude mít kmitavý charakter, ale bude stabilní.

3.3.4 Luenbergerův stavový estimátor

Použití stavového regulátoru, například LQR, je založeno na stavové zpětné vazbě, která předpokládá znalost celého stavového vektoru systému. Zkonstruovaná soustava je osazena pouze dvěma snímači úhlu. Další dva stavy úhlových rychlostí nejsou měřeny. To znamená, že je nutné zbylé stavy odhadnout z dostupných měření.

Úhlová rychlost je definována jako derivace úhlu podle času. Jelikož mikroprocesor pracuje v diskrétním čase, derivaci je aproximována diferencí. Tento přístup však vede k problémům, protože derivace zesiluje vysokofrekvenční složky signálu, tedy i měřicí šum snímačů. Příklad toho lze pozorovat na obrázku 3.4. Vypočítané hodnoty se tedy stávají nepřesnými, zpětný stavový regulátor nedostává přesnou informaci o poloze, a tedy nezvládne správně reagovat na regulační odchylku. Existují ale i jiné přístupy k tomu, jak určit chybějící stavy.

Problém lze řešit pomocí stavových pozorovatelů, jako je Luenbergerův estimátor ("Luenberger observer"). Jeho princip je založen na využití matematického modelu systému popsaného stavovými rovnicemi, přičemž předpokládá znalost systémových matic A , B a C . Pokud je známé chování systému a lze měřit některé stavy, na jejichž základě je možné odhadnout celý stavový vektor, aniž by bylo nutné přímo měřit všechny jeho složky.

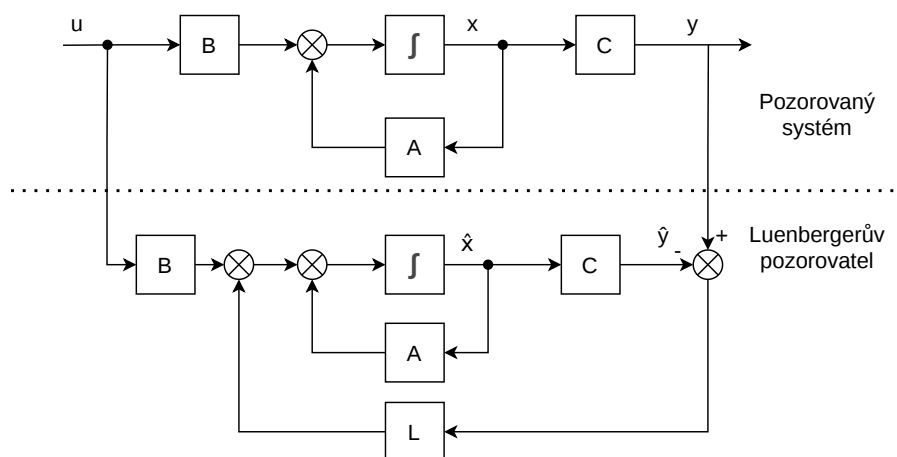
Rozšířením stavové rovnice systému $\dot{x} = Ax + Bu$ o korekční člen lze získat rovnici Luenbergerova pozorovatele.

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - \hat{y}) \quad (3.25)$$

Na základě této rovnice lze zapojit blokové schéma estimátoru:

Na základě schématu lze pozorovat, že $\hat{x}$ představuje odhad jednotlivých stavů. $\hat{y}$ je odhad výstupu systému, který je odečten od reálného výstupu pro korekci modelu. Když je pozorovatel použit v kombinaci se zpětným stavovým regulátorem, matice K bude násobit stavový vektor $\hat{x}$.

Je patrné, že metoda vyžaduje návrh matice L . Postup pro získání koeficientů je prakticky totožný s návrhem stavové zpětné vazby, tedy metodou rozložení pólů



Obr. 3.5: Luenbergerův stavový estimátor

nebo LQR. Dále je důležité zajistit, aby póly estimátoru ležely dále v levé komplexní polorovině než póly uzavřené regulační smyčky. Díky tomu je dynamika estimátoru rychlejší než dynamika samotného systému a odhad stavů se dostatečně rychle přibližuje skutečným hodnotám.[34]

Póly estimátoru byly zvoleny následovně:

$$P = \{-700; -750; -701; -753\}$$

Realizovaný Luenbergerův pozorovatel na matematickém modelu Furuta pendulum je zobrazen v příloze B.1, v řízení je zahrnuta i zpětná stavová regulace.

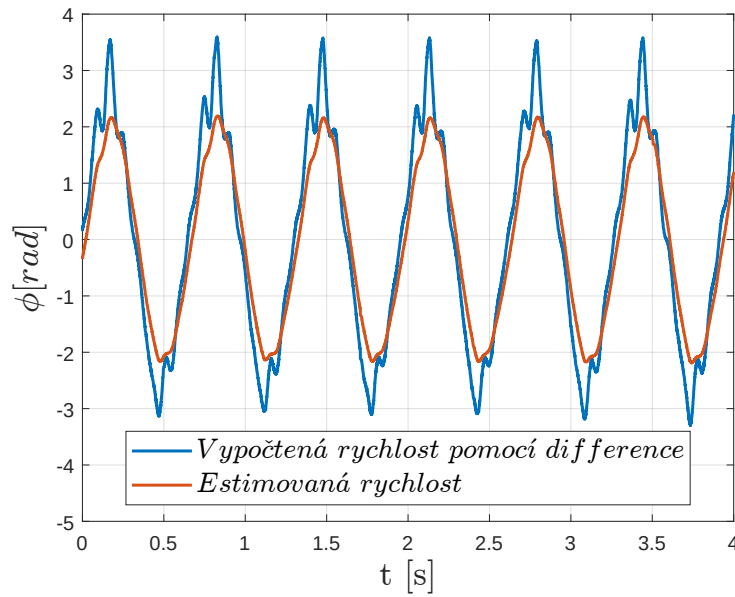
Pro ověření správné funkce estimátoru lze využít obrázky 3.5 a 3.6, které zobrazují průběhy neměřených stavů, konkrétně úhlových rychlostí ramene a kyvadla na reálné řízené soustavě. Výsledky jsou porovnány se stavy, které byly rekonstruovány pomocí derivace měřených úhlů. Z obrázku 3.6 je patrné výrazné zesílení šumu enkodéru při použití derivace. To potvrzuje, že použití takového signálu ve stavové zpětné vazbě by vedlo k nevhodnému akčnímu zásahu.

Dále můžeme usoudit, že estimátor je nastaven správně, protože stavy konvergují velmi rychle a systém reaguje téměř bez zpoždění.

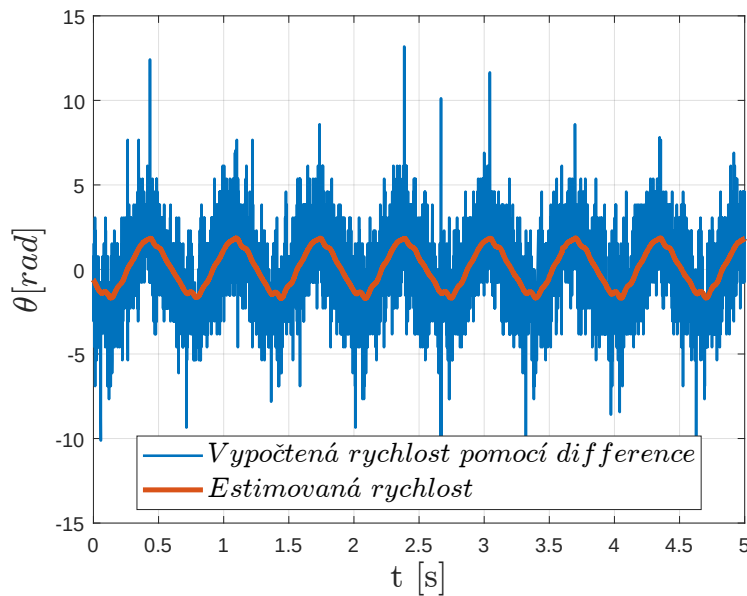
Průběhy měřených úhlů nejsou ukázány, protože póly estimátoru byly zvoleny dostatečně rychlé a odhadované úhly proto téměř přesně kopírují jejich měřené hodnoty.

3.4 Implementace řízení v jazyce C

Návrhové metody byly provedeny na matematickém modelu v matlabu. Reálné řízení jen navržené parametry přebírá, je tedy nezbytné mít dostatečně kvalitně navržený



Obr. 3.6: Srovnání úhlové rychlosti ramene pomocí estimátoru a derivace



Obr. 3.7: Srovnání úhlové rychlosti kyvadla pomocí estimátoru a derivace

model inverzního kyvadla a simulaci řízení.

Řídicí algoritmus vznikl rozšířením programu pro FOC. Jelikož výstupem stavové zpětné vazby je moment a vstupem momentové smyčky FOC je požadovaný moment motoru, jsou oba systémy vzájemně kompatibilní. Moment stačí podělit momentovou konstantou motoru a výsledkem operace je proud I_q , který se tak stává požadovanou hodnotou w_{I_q} . Celá zpětná stavová regulace byla realizována v přeru-

šení ADC převodníku, kde se nachází také celý FOC algoritmus.

Definice úhlů

Nejprve bylo ale potřeba rozšířit dosavadní CubeMX konfiguraci o SPI komunikaci s magnetickým enkodérem, který snímá úhel kyvadla. Pro správnou konfiguraci je potřeba nastavit $CPOL = 1$ a $CPHA = 0$, tedy mód 1. Ke komunikaci je využito 16 bitů, z toho 15. bit je paritní a 14. bit je informace o chybě, zbylých 14 bitů je využito pro informaci o poloze. Využité piny jsou uvedeny v tabulce 3.2.

Tab. 3.2: Použité piny pro SPI

Periferie	Pin	Funkce	Význam
SPI2	PB12	SPI2_CS	Chip Select - začátek komunikace
	PB13	SPI2_SCK	Clock - hodinový signál
	PB14	SPI2_MISO	Master In Slave Out
	PB15	SPI2_MOSI	Master Out Slave In

Snímač měří v rozsahu 0 až 2π , to je pro aplikaci nevhodné, protože horní poloha kyvadla odpovídá hodnotě 0. Bez úpravy by proto docházelo k velkým skokům úhlu okolo horní polohy. Z tohoto důvodu byl proveden přepoččet do rozsahu $\pm\pi$.

Dále bylo potřeba definovat orientaci úhlů tak, aby odpovídala matematickému modelu, jinak by řízení nefungovalo správně. Při aplikaci kladného momentu (+) na model se úhly chovají následovně: ϕ začne růst do kladných hodnot (+), přičemž pohyb probíhá po směru hodinových ručiček. Úhel θ začne růst do kladných hodnot (+), přičemž pohyb je proti směru hodinových ručiček. Hodnota magnetického enkodéru (úhel θ) však roste do kladných hodnot (+) při pohybu po směru hodinových ručiček. Tedy přesně naopak, než předpokládá matematický model. Z tohoto důvodu bylo nutné hodnotu θ násobit konstantou -1.

Diskrétní estimátor

Při pokusu o zpětnou stavovou regulaci s dopočítáním úhlových rychlostí pomocí derivací došlo k již zmíněnému problému s nepřesným signálem kvůli šumu, viz. obrázek 3.7. Bylo tedy přistoupeno k použití Luenbergerova estimátoru.

Protože v estimátoru figuruje integrátor, je při reálném řízení využít diskrétní stavový regulátor. Samotný pozorovatel musí mít póly rychlejší než póly uzavřeného systému. To ale neznamená, že musí být rychlejší než FOC smyčka; je vhodné zvolit menší vzorkovací frekvenci estimátoru než algoritmu vektorového řízení. Vzorkovací frekvence byla zvolena 10 kHz. Implementace diskrétního pozorovatele je v příloze

B.1. Návrh matice L byl popsán v kapitole 3.3.4. Tedy matice A , B , C a L byly použity stejné jako při návrhu v prostředí MATLAB.

Posledním krokem bylo jednotlivé stavy $\hat{x}$ roznásobit maticí zpětné vazby K a sečíst. Vnikl tak moment, který podělením momentovou konstantou motoru představuje velikost proudu Iq . Proud byl zasaturován na strop výkonové desky, což jsou 2 ampéry. Tento proud vstupuje jako požadovaná hodnota do PI regulátoru proudové smyčky algoritmu FOC.

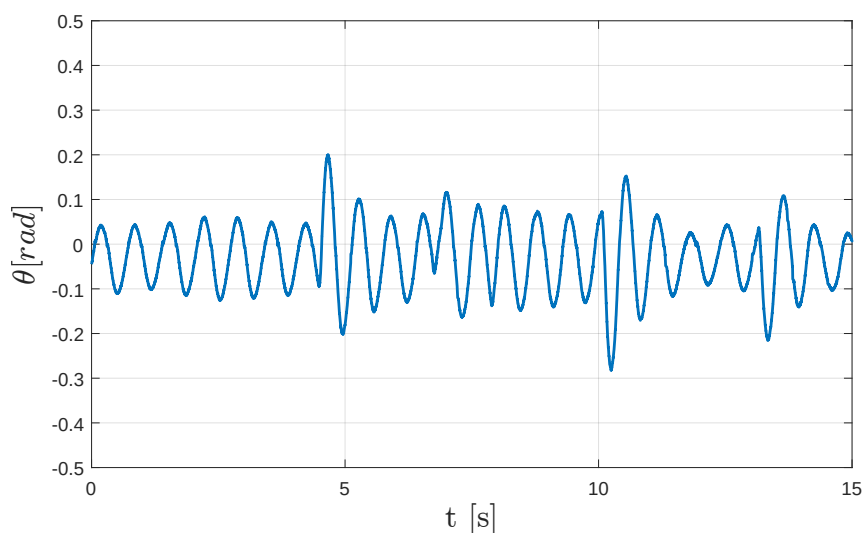
3.5 Experimentální výsledky a porovnání s modelem

Cílem práce bylo stabilizovat inverzní rotační kyvadlo (úhel θ) v horní poloze a rameno (úhel ϕ) regulovat do počáteční polohy. Jako řešení byly navrženy dva přístupy, a to pomocí metody umístění pólů a LQR. Úspěšnou stabilizaci kyvadla v horní poloze prezentují následující grafy 3.8 až 3.11. Systém při všech nastaveních vykazuje trvalé oscilace, které jsou zapříčiněny více faktory. Jelikož se jedná o reálný systém, přirozeně obsahuje dopravní zpoždění, což, jak je známo z teorie řízení, přidává fázi a celkově zhoršuje stabilitu systému. Existují metody kompenzace dopravního zpoždění, ale těmi se tato práce nezabývá.

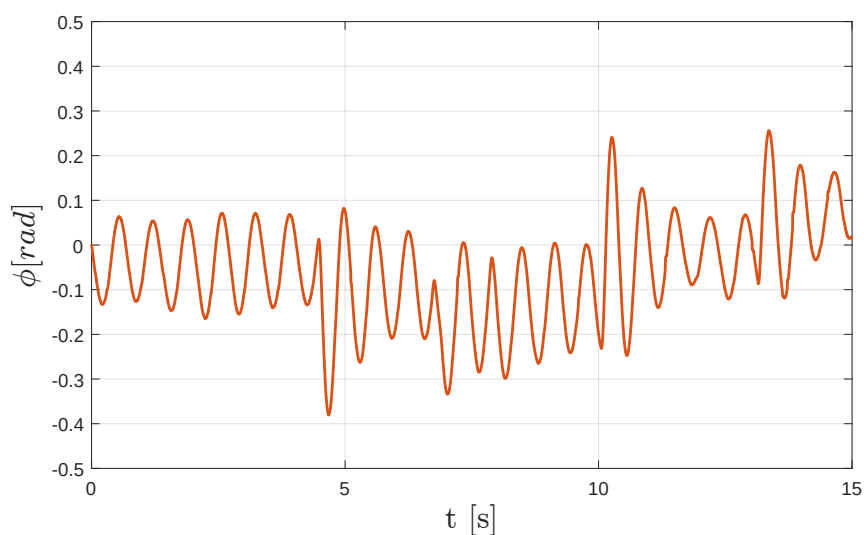
Dalším rušivým faktorem je kabeláž snímače úhlu kyvadla, která může do systému přidávat efekt pružiny nebo tření. Významnou roli hrají také nelineární třecí jevy, především v okolí bodu zastavení. Tyto a další důvody způsobují, že matematický model kyvadla nikdy nebude stoprocentně odpovídat realitě. Touto nepřesností je dále zasažen estimátor stejně jako zpětný stavový regulátor, které jsou založeny na matematickém modelu. Výsledný akční zásah tak nebude nikdy zcela přesný, což se následně projevuje na kvalitě regulace.

Výraznější špičky signálu odpovídají experimentálním testům stability, při kterých bylo do kyvadla mechanicky zasaženo.

Průběhy na obrázcích 3.8 a 3.9 zobrazují stabilizaci okolo horní polohy pomocí regulátoru navrženého metodou umístění pólů. Systém úspěšně udržuje rameno ve vzpřímené poloze i při vnějším zásahu. Kyvadlo kmitá přibližně $\pm 6^\circ$, což se dá považovat za kvalitní regulaci. Dále lze pozorovat, že systém má obtíže regulovat nulovou polohu ramene, což je způsobeno experimentálním nastavením pólů matice K .

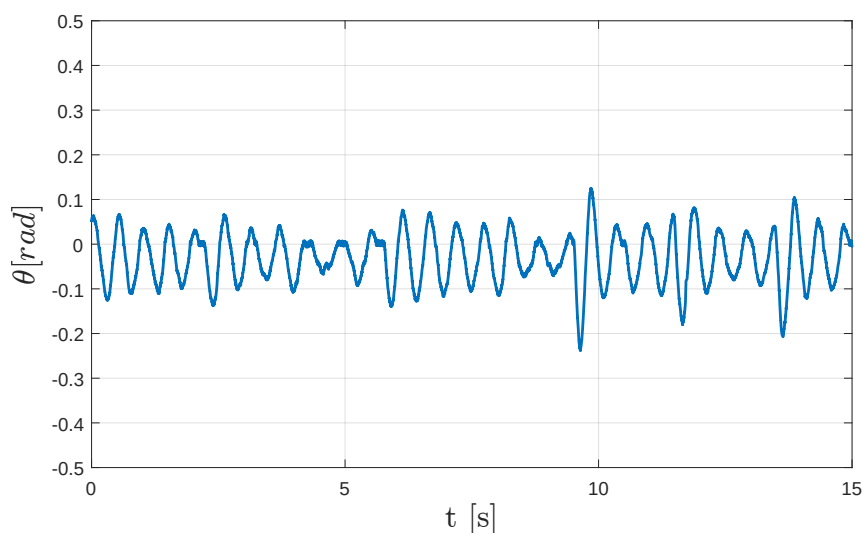


Obr. 3.8: Průběh úhlu θ pomocí návrhové metody umístěním pólů

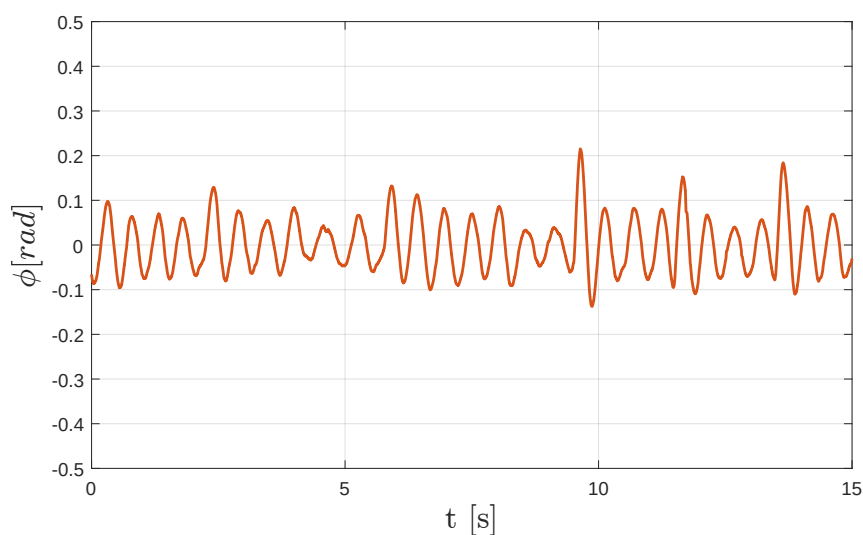


Obr. 3.9: Průběh úhlu ϕ pomocí návrhové metody umístěním pólů

Úhly na obrázcích 3.10 a 3.11 prezentují stabilizaci pomocí regulátoru navrženého metodou LQR. Regulaci lze považovat za agresivnější, reakce na externí zásah je rychlejší, ale nevýhodou jsou přítomné vysokofrekvenční oscilace. Stabilizace okolo horní polohy je tedy úspěšná stejně jako regulace ramena do počáteční polohy.

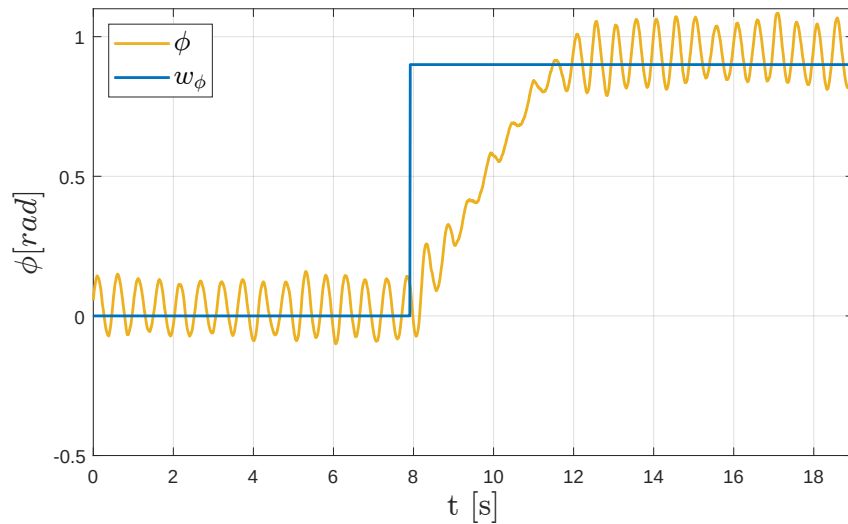


Obr. 3.10: Průběh úhlu θ pomocí návrhové metody LQR

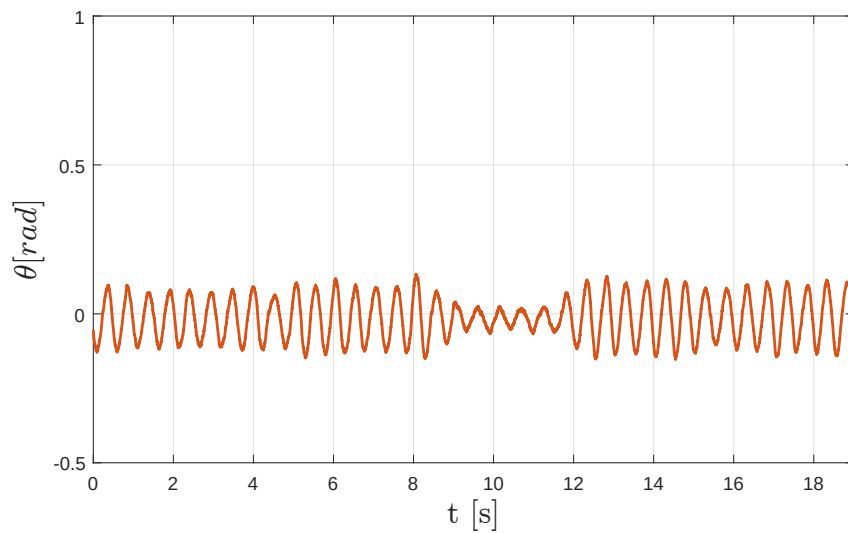


Obr. 3.11: Průběh úhlu ϕ pomocí návrhové metody LQR

Obrázky 3.12 a 3.13 potvrzují další úspěšně splněný aspekt úlohy, kdy při změně požadovaného úhlu ϕ se rameno přemístí na novou pozici a přitom zvládne stabilizovat kyvadlo. Při změně požadovaného úhlu v čase $t = 8$ s se rameno plynule přesune na novou pozici, přičemž přechodový děj trvá přibližně 2 sekundy. Během pohybu ramene dochází na grafu 3.13 k dočasnému utlumení kmitů kyvadla. Regulační odchylka úhlu θ se stabilně udržuje v úzkém pásmu kolem rovnovážné polohy. Výsledky tak potvrzují schopnost navrženého algoritmu stabilizovat nelineární systém i během dynamických změn.



Obr. 3.12: Průběh úhlu ϕ při změně požadovaného úhlu ramene



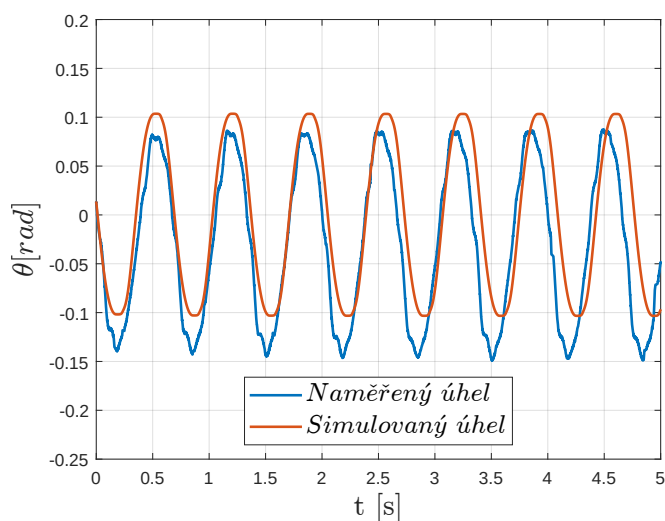
Obr. 3.13: Průběh úhlu θ při změně požadovaného úhlu ramene

Na obrázcích 3.14 a 3.15 je prezentováno porovnání průběhů úhlů matematického nelineárního modelu Furuta pendulum a reálně řízené soustavy. Z grafů lze pozorovat shodu v periodě a frekvenci kmitů u obou sledovaných úhlů, tedy kyvadla θ a ramene ϕ .

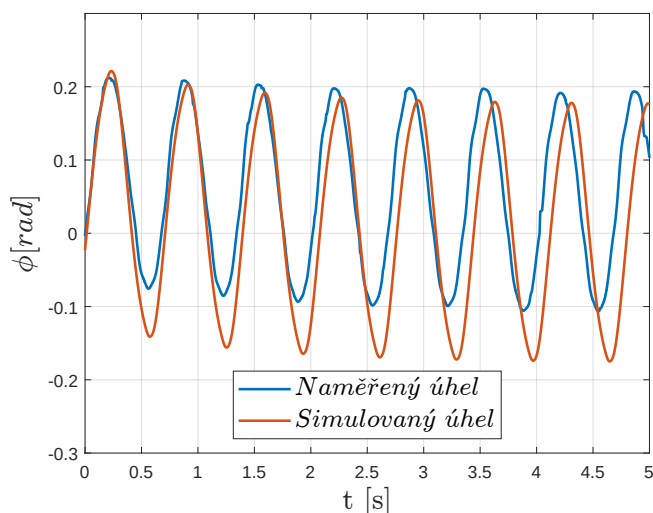
Matematický model se však postupem času od reálné soustavy odchyluje, protože představuje idealizovaný systém s pravidelným průběhem kmitů, zatímco reálná soustava je ovlivněna řadou nelineárních a obtížně modelovatelných jevů, které byly zmíněny na začátku této kapitoly.

Stěžejní roli hrají také počáteční podmínky systému. Reálná soustava se při každém řízení nachází v mírně odlišném počátečním stavu, který je obtížné v simulaci přesně definovat. Chování systému je na počátečních podmínkách závislé, a i proto dochází k postupnému rozchodu simulace a reálného systému.

I přes uvedené nepřesnosti lze pozorovat, že matematický model dostatečně odpovídá reálnému systému. V případě, kdy dochází k odchylce mezi simulací a reálnou soustavou, pomáhá tuto nepřesnost kompenzovat stavový estimátor pomocí korekčního členu: $L(y - \hat{y})$. Díky této korekci je možné průběžně upravovat odhad stavových veličin podle skutečně naměřených hodnot systému.



Obr. 3.14: Srovnání úhlu kyvadla modelu s realitou



Obr. 3.15: Srovnání úhlu ramene modelu s realitou

Závěr

Cílem bakalářské práce bylo navrhnout a implementovat vektorové řízení BLDC motoru a následně jej využít pro stabilizaci rotačního invertovaného kyvadla. Součástí práce bylo vytvoření matematického modelu motoru i samotného kyvadla, návrh stavového regulátoru a estimátoru stavových veličin a realizace algoritmů v prostředí MATLAB/Simulink a jazyce C.

První část bakalářské práce pojednávala o problematice BLDC motorů a jejich řízení. Hlavní pozornost byla věnována algoritmu FOC, který byl následně použit pro řízení motoru. Další část teoretického rozboru zahrnovala řešení na soustavy malé velikosti vhodné pro výuku teorie řízení. Součástí byl popis Furuta pendulum, jehož řízením se tato práce zabývá.

V druhé části práce byl vytvořen matematický model BLDC motoru a navržena momentová regulační smyčka FOC. Dále bylo úspěšně implementováno vektorové řízení BLDC motoru na platformě STM32 v programovacím jazyce C a Simulinku. Jednotlivé implementace algoritmu byly ověřeny a jejich odezvy byly porovnány pomocí přechodové charakteristiky. Naměřené průběhy potvrdily dostatečně rychlou a stabilní regulaci momentu motoru.

Třetí kapitola byla věnována sestavení laboratorního přípravku a identifikaci jeho parametrů. Dále byl navržen matematický model a stavové matice. Pomocí simulace byly na modelu otestovány regulátory navržené metodou umístění pólů a metodou LQR. Součástí řídicího algoritmu byl také navržen Luenbergerův estimátor neměřených stavů.

Navržené algoritmy byly následně implementovány na reálný systém. Výsledky potvrdily správnost algoritmu stabilizace kyvadla v horní nestabilní poloze a regulace polohy ramene do počátku. Reálná soustava má trvalé oscilace a odchylky oproti matematickému modelu. Tyto nepřesnosti jsou způsobeny zejména nelineárním třením, dopravním zpožděním a mechanickými nepřesnostmi.

Přesto lze konstatovat, že zadání práce bylo splněno a navržený systém je dostatečně rychlý pro regulaci proudové smyčky a je schopný stabilního řízení Furuta pendulum. Možným pokračováním práce by byla přesnější identifikace parametrů systému, dále doplnění swing-up algoritmu, aplikace Kalmanova filtru jako estimátoru nebo prozkoumání využití dalších pokročilejších metod řízení.

Literatura

- [1] HERMAN, Stephen L. *Industrial Motor Control*. 7. vydání. Clifton Park, NY: Delmar, Cengage Learning, 2014. ISBN 13: 978-1-133-69180-8.
- [2] MILLETT, Pete. Brushless vs. Brushed DC Motors: When and Why to Choose One Over the Other. Online. Dostupné z: https://media.monolithicpower.com/mps_document/2/0/2021-brushless-vs-brushed-dc-motors-when-and-why-to-choose-one-over-the-other_1.0.pdf. [cit. 2026-05-01].
- [3] *Haredataelectronics* [online]. In: . 2020 [cit. 2026-5-01]. Dostupné z: <https://www.haredataelectronics.co.uk/brushed-dc-motors-vs-brushless-dc-motors>
- [4] NEBORÁK, Ivo. *Modelování a simulace elektrických regulovaných pohonů*. Ostrava, 2002. Habilitační práce. Vysoká škola báňská - Technická univerzita Ostrava.
- [5] *What is BLDC motor*. Online. In: ABLIC. Dostupné z: <https://www.ablic.com/en/semicon/applications/bldc-motor/what-bldc-motor/>. [cit. 2026-05-01].
- [6] BOLDEA, Ion a NASAR, Syed A. *Electric Drives*. Online. 3. Taylor & Francis Group, 2016. ISBN 9781498748216. Dostupné z: <https://ebookcentral.proquest.com/lib/vutbrno/detail.action?docID=5209730&pq-origsite=primo#>. [cit. 2026-05-01].
- [7] BHIM, Singh a SANJEEV, Singh. State-of-art on permanent magnet brushless DC motor drives. Online. *Journal of Power Electronics*. 2009. ISSN 1598-2092. Dostupné z: <https://repository.ias.ac.in/106485/1/9-1.pdf>. [cit. 2026-05-02].
- [8] AKIN, Bilal; BHARDWAJ, Manish a WARRINER, Jon. Trapezoidal Control of BLDC Motors Using Hall Effect Sensors. Online. *Texas Instruments*. 2015. Dostupné z: https://www.ti.com/lit/an/sprabz4/sprabz4.pdf?ts=1735501036707&ref_url=https%253A%252F%252Fwww.google.com%252F. [cit. 2026-05-03].
- [9] RAMAMOORTHY, Ramesh T; LARIMORE, Brett a BHARDWAJ, Manish. Sensored Field Oriented Control of 3-Phase Permanent Magnet Synchronous Motors Using TMS320F2837x. Online. *Texas Instruments*. 2022. Dostupné z: https://www.ti.com/lit/an/sprabz0a/sprabz0a.pdf?ts=1735563293177&ref_url=https%253A%252F%252Fwww.google.com%252F . [cit. 2026-05-05].

- [10] TEXAS INSTRUMENTS. *Precision labs series: Brushless-DC motors*. Online. Texas Instruments. 2022. Dostupné z: <https://www.ti.com/video/series/precision-labs/ti-precision-labs-brushless-dc-motors.html>. [cit. 2026-05-05].
- [11] TÜRKMEN, A.; KORKUT, M. Y.; ERDEM, M.; GÖNÜL, Ö. a SEZER, V. Design, implementation and control of dual axis self balancing inverted pendulum using reaction wheels. Online. Dostupné z: <https://ieeexplore.ieee.org/abstract/document/8266226>. [cit. 2026-05-05].
- [12] BalancingStick. Online. In: Github. Dostupné z: <https://github.com/gregoryxie/BalancingStick>. [cit. 2026-05-06]
- [13] QUANSER. Ball and Beam Experiment. Online. Dostupné z: <https://aarnphm.xyz/thoughts/university/twenty-three-twenty-four/sfwr-3dx4/lab4/BallBeamUserManual.pdf>. [cit. 2026-05-06].
- [14] SHAH, Mustansar; ALI, Rahat a MALIK, Fahad Mumtaz. Control of Ball and Beam with LQR Control Scheme using Flatness Based Approach. Online. Dostupné z: <https://ieeexplore.ieee.org/document/8610968>. [cit. 2026-05-06].
- [15] KUMAR, E. Vinodh a JEROME, Jovitha. LQR based Optimal Tuning of PID Controller for Trajectory Tracking of Magnetic Levitation System. Online. Dostupné z: <https://www.sciencedirect.com/science/article/pii/S1877705813016111>. [cit. 2026-05-05].
- [16] TKÁČIK, Tomáš; TKÁČIK, Milan; JADLOVSKÁ, Slávka a JADLOVSKÁ, Anna. Design of Aerodynamic Ball Levitation Laboratory Plant. Online. Dostupné z: <https://www.mdpi.com/2227-9717/9/11/1950>. [cit. 2026-05-06].
- [17] GÄFVERT, Magnus. Modelling the Furuta Pendulum. Online. Dostupné z: <https://lucris.lub.lu.se/ws/files/4453844/8727127.pdf>. [cit. 2026-05-05].
- [18] CAZZOLATO, Ben S. a PRIME, Zebb. On the dynamics of the Furuta pendulum. Online. Dostupné z: <https://pdfs.semanticscholar.org/27a3/f8fc5954c5a1506f026b0c3731d36bc474.pdf>. [cit. 2026-05-06].
- [19] QUANSER. Rotary Inverted Pendulum. Online. Dostupné z: <https://www.quanser.com/products/rotary-inverted-pendulum/>. [cit. 2026-05-06].

- [20] MATHWORKS. *Simulink*. Online. 2024. Dostupné z: <https://www.mathworks.com/products/simulink.html>. [cit. 2026-05-18].
- [21] ST. *STM32Cube initialization code generator*. Online. Dostupné z: <https://www.st.com/en/development-tools/stm32cubemx.html>. [cit. 2026-05-18].
- [22] ST. *Integrated Development Environment for STM32*. Online. Dostupné z: <https://www.st.com/en/development-tools/stm32cubeide.html>. [cit. 2026-05-18].
- [23] ST. *Monitoring tool to test STM32 applications at run-time*. Online. Dostupné z: <https://www.st.com/en/development-tools/stm32cubemonitor.html>. [cit. 2026-05-18].
- [24] STMICROELECTRONICS. *STSPIN830: Compact and versatile three-phase and three-sense BLDC motor driver*. 2019. Dostupné také z: <https://www.st.com/en/motor-drivers/stspin830.html#documentation>.
- [25] STMICROELECTRONICS. *STM32G4 Nucleo-64 boards (MB1367): User manual*. 2021. Dostupné také z: <https://st.com/en/evaluation-tools/nucleo-g474re.html#documentation>.
- [26] STMICROELECTRONICS. *STM32G4 series advanced Arm®-based 32-bit MCUs: Reference manual*. 2024. Dostupné také z: <https://www.st.com/en/microcontrollers-microprocessors/stm32g474re.html#documentation>.
- [27] STMICROELECTRONICS. *X-NUCLEO-IHM16M1: Three-phase brushless DC motor driver expansion board based on STSPIN830 for STM32 Nucleo*. 2024. Dostupné také z: <https://www.st.com/en/ecosystems/x-nucleo-ihm16m1.html#documentation>.
- [28] DFA68M024035-E – Brushless DC flat motor with encoder. Online. Nanotec. Dostupné z: <https://www.nanotec.com/eu/en/products/9888-dfa68m024035-e>. [cit. 2026-05-10].
- [29] ST. *Introduction to timers for STM32 MCUs*. Online. 2024. Dostupné také z: https://www.st.com/resource/en/application_note/an4013-introduction-to-timers-for-stm32-mcus-stmicroelectronics.pdf.
- [30] MAGDY, Khaled. : *STM32 ADC Tutorial + ADC Examples [Ultimate Guide]*. Online. Dostupné z: <https://deepbluembedded.com/stm32-adc-tutorial-complete-guide-with-examples/>. [cit. 2026-05-01].

- [31] BLAHA, Petr a VAVŘÍN, Petr. Řízení a regulace I. Online. In: . Dostupné z: https://www.uamt.fekt.vut.cz/richter/vyuka/1011_BRR1/texty/brr1.pdf. [cit. 2026-05-25].
- [32] AMS OSRAM. *AS5147P: 14-Bit On-Axis Magnetic Rotary Position Sensor with 12-Bit Binary Incremental Pulse Count and for 28krpm High Speed Capability*. 2016. Dostupné také z: <https://device.report/manual/9004749>. [cit. 2026-05-01].
- [33] MATHWORKS. What is Pole Placement (Full State Feedback) | State Space, Part 2. Online. In: Youtube. 2019. Dostupné z: <https://www.youtube.com/watch?v=FXSpHy8LvmYt=610s>. [cit. 2026-05-18].
- [34] VÍTEČEK, Antonín a VÍTEČKOVÁ, Miluše. ZPĚTNOVAZEBNÍ ŘÍZENÍ MECHATRONICKÝCH SYSTÉMŮ. Online, Učební text. Ostrava: VŠB – Technická univerzita Ostrava, 2013. Dostupné z: <https://books.fs.vsb.cz/ZRMS/zpetnovazebni-rizeni-mechatronickych-systemu.pdf>. [cit. 2026-05-18].
- [35] RABIK MOHAMED ISMAIL, Mohamed. An Efficient Coordinated Observer LQR Control in a Platoon of Vehicles for Faster Settling Under Disturbances. Online. Dostupné z: <https://www.mdpi.com/2032-6653/16/1/28>. [cit. 2026-05-19].
- [36] MATHWORKS. What Is Linear Quadratic Regulator (LQR) Optimal Control? | State Space, Part 4. Online. In: Youtube. 2019. Dostupné z: https://www.youtube.com/watch?v=E_RDCFOLJx4t=649s. [cit. 2026-05-18].

Seznam symbolů a zkratek

DC	Direct current - Stejnosměrný proud
BLDC	Brushless Direct current motor - Stejnosměrný bezkartáčový motor
IGBT	Insulated Gate Bipolar Transistor - Bipolární tranzistor s izolovaným hradlem
PWM	Pulse Width Modulation - Pulzně šířková modulace
BEMF	Counter-electromotive force - Zpětná elektromotorická síla
FOC	Field-Oriented Control - Vektorové řízení
TIM	Timer - Časovač
TRGO	Trigger Output - Spouštěcí výstup
ADC	Analog-to-digital converter - Analogově-digitální převodník
SYSCLK	System clock - Systémový hodinový signál
PSC	Prescaler - Předdělička
ARR	Auto-Reload Register - Strop časovače
RCR	Repetition Counter Register - opakovací registr
GND	Ground - zem, referenční vodič
SPI	Serial Peripheral Interface - sériové periferní rozhraní
MOSI	Master out, slave in - Výstup z mastr do slave zařízení
MISO	Master in, slave out - Výstup ze slave do mastr zařízení
SS	Slave select -výběr slave zařízení
SCLK	Serial clock - seriový hodinový signál
CPOL	Clock Polarity – Polarita hodinového signálu
CPHA	Clock Phase – Fáze hodinového signálu
MCU	Microcontroller Unit - mikrokontrolér
ARM	Advanced RISC Machine – Architektura procesoru

FPU	Floating Point Unit – Jednotka pro výpočty s desetinou čárkou
LQR	Linear–quadratic regulator – Lineárně kvadratický regulátor

symboly:

I_a	Proud fází a	[A]
I_b	Proud fází b	[A]
I_c	Proud fází c	[A]
I_α	Proud alfa v souřadnicovém systému α,β	[A]
I_β	Proud beta v souřadnicovém systému α,β	[A]
I_d	Proud tokotvorné složky	[A]
I_q	Proud momentotvorné složky	[A]
U_a	Napětí fáze a	[U]
U_b	Napětí fáze b	[U]
U_c	Napětí fáze c	[U]
U_α	Napětí alfa v souřadnicovém systému α,β	[U]
U_β	Napětí beta v souřadnicovém systému α,β	[U]
U_d	Napětí tokotvorné složky	[U]
U_q	Napětí momentotvorné složky	[U]
φ	Úhel natočení rotoru	[°]
R_s	Mezifázový Odpor	[Ω]
L_d	Indukčnost tokotvorné složky	[H]
L_q	Indukčnost momentotvorné složky	[H]
N_p	Počet pólových dvojic motoru	[-]
ψ_f	Magnetický tok	[Wb]
J	Moment setrvačnosti	[kg · m ²]

e	Regulační odchylka	[A]
ω	Úhlová rychlost	[rad/s]
M	Točivý moment	[Nm]
$\dot{x}$	Stavový vektor	[-]
$\hat{x}$	Estimovaný stavový vektor	[-]
A	Systémová matice	[-]
B	Matice vstupů	[-]
C	Matice výstupů	[-]
D	Matice přímých vazeb	[-]
K	Matice zesílení zpětné stavové vazby	[-]
L	Matice zesílení zpětné Luenbergerova estimátoru	[-]
P	Vektor pólů systému	[-]
Q	Matice LQR regulátoru pro penalizaci stavů	[-]
R	Matice LQR regulátoru pro penalizaci akčního zásahu	[-]

Seznam příloh

A Implementace řízení BLDC motoru	70
A.1 Nastavení a povolení pinů pro PWM	70
A.2 Výpočet mechanického a elektrického úhlu	70
A.3 Měření proudů pomocí ADC	71
A.4 Diskrétní PI regulátor	71
A.5 Implementace vektorového řízení	72
A.6 Implementace vektorového řízení v Simulinku	73
B Furuta Pendulum	74
B.1 Simulace řízení Furuta pendulum s estimátorem	74
B.2 Implementace Luenbergerova pozorovatele	75
B.3 Sestrojené inverzní rotační kyvadlo	76
C Obsah elektronické přílohy	77

A Implementace řízení BLDC motoru

A.1 Nastavení a povolení pinů pro PWM

Výpis A.1: Nastavení portů PWM pro výkonovou desku IHM07M1

```
//Timer start
HAL_TIM_Encoder_Start(&htim8, TIM_CHANNEL_ALL);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
//Enable Phases
HAL_GPIO_WritePin(Enable_U_GPIO_Port, Enable_U_Pin,
GPIO_PIN_SET);
HAL_GPIO_WritePin(Enable_V_GPIO_Port, Enable_V_Pin,
GPIO_PIN_SET);
HAL_GPIO_WritePin(Enable_W_GPIO_Port, Enable_W_Pin,
GPIO_PIN_SET);
HAL_GPIO_WritePin(Enable_F_GPIO_Port, Enable_F_Pin,
GPIO_PIN_SET);
//Launch ADC with interrupt
HAL_ADCEx_InjectedStart_IT(&hadc1)
```

A.2 Výpočet mechanického a elektrického úhlu

Výpis A.2: Přepočítání hodnoty enkodéru na úhel natočení hřídele

```
//Angle
counter = __HAL_TIM_GET_COUNTER(&htim8);
position = (6.2831853f / 16384.0f) * counter;
//Electric angle
theta = fmodf((POLE_PAIRS*(position - offset_motor)) + M_PI,
6.2831853f) - M_PI;
```

A.3 Měření proudů pomocí ADC

Výpis A.3: Přepočítání ADC hodnot na proudy v jednotlivých fázích

```
//ADC hodnota
adc_ch_u = HAL_ADCEx_InjectedGetValue(hadc,
ADC_INJECTED_RANK_1);
adc_ch_v = HAL_ADCEx_InjectedGetValue(hadc,
ADC_INJECTED_RANK_2);
adc_ch_w = HAL_ADCEx_InjectedGetValue(hadc,
ADC_INJECTED_RANK_3);
//ADC value to voltage
float U1 = adc_ch_u * (3.3 / 4096);
float U2 = adc_ch_v * (3.3 / 4096);
float U3 = adc_ch_w * (3.3 / 4096);
//Voltage to current
Ia = (U1 - offset_u)/(gain * shunt_r);
Ib = (U2 - offset_v)/(gain * shunt_r);
Ic = (U3 - offset_w)/(gain * shunt_r);
```

A.4 Diskrétní PI regulátor

Výpis A.4: Implementace diskrétního PI regulátoru

```
float32_t PI_controler(arm_pid_instance_f32 *S, float32_t error) {
    //S->state[0] = error;

    float32_t proportion = S->Kp * error;
    float32_t integral = S->Ki * error * TIME_STEP;
    //Storage of integrated value
    float32_t integral_sum = S->state[0] + integral;
    S->state[0] = integral_sum;

    float32_t out = proportion + integral_sum;
    return out;
}
```

A.5 Implementace vektorového řízení

Výpis A.5: Regulační smyčka vektorového řízení v D-Q souřadnicovém systému

```
sin_val = arm_sin_f32(theta);      1
cos_val = arm_cos_f32(theta);      2

                                     3

arm_clarke_f32(Ia, Ib, &IAIpha, &IBeta);      4
arm_park_f32(IAIpha, IBeta, &Id, &Iq, sin_val, cos_val);      5

                                     6

//Error      7
ed = w_id - Id;      8
eq = w_iq - Iq;      9

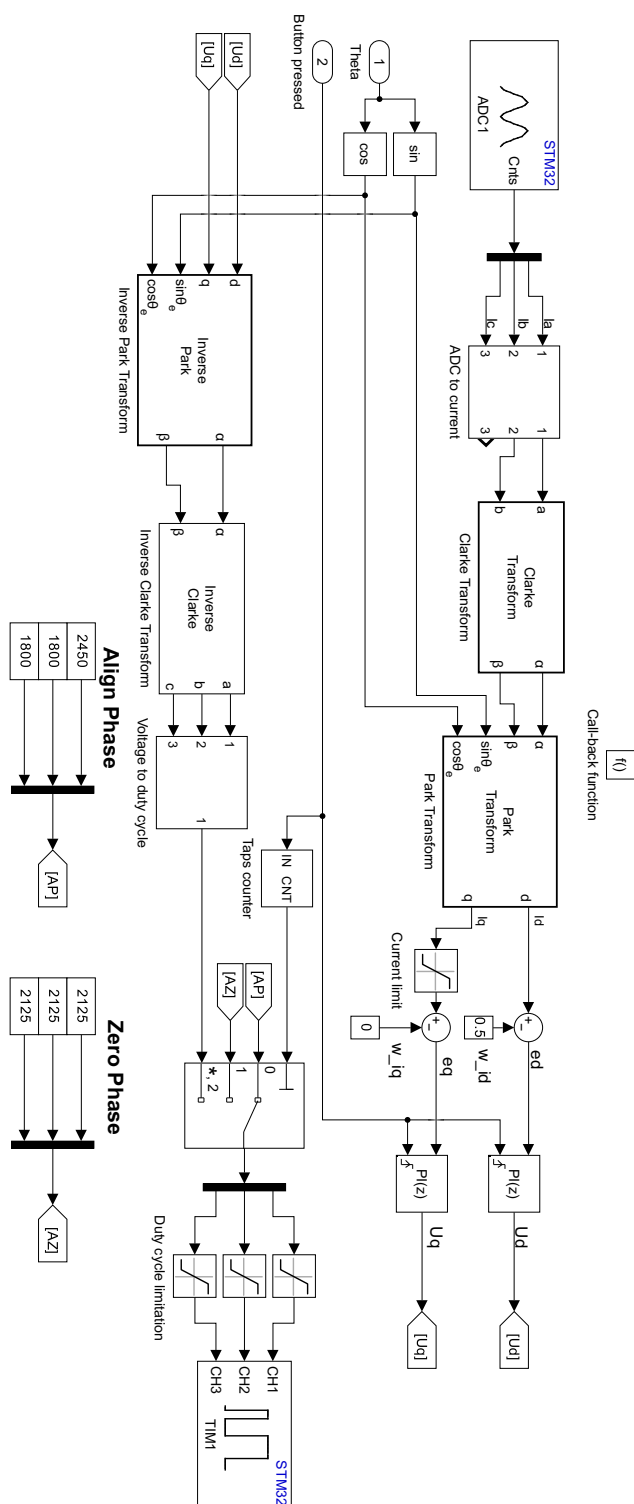
//Action     10
Ud = PI_controler(&pi_d, ed);      11
Uq = PI_controler(&pi_q, eq);      12

                                     13

arm_inv_park_f32(Ud, Uq, &UAlfa, &UBeta, sin_val, cos_val);      14
ReverseClarke(UAlfa, UBeta, &Ua, &Ub, &Uc);      15
```

A.6 Implementace vektorového řízení v Simulinku

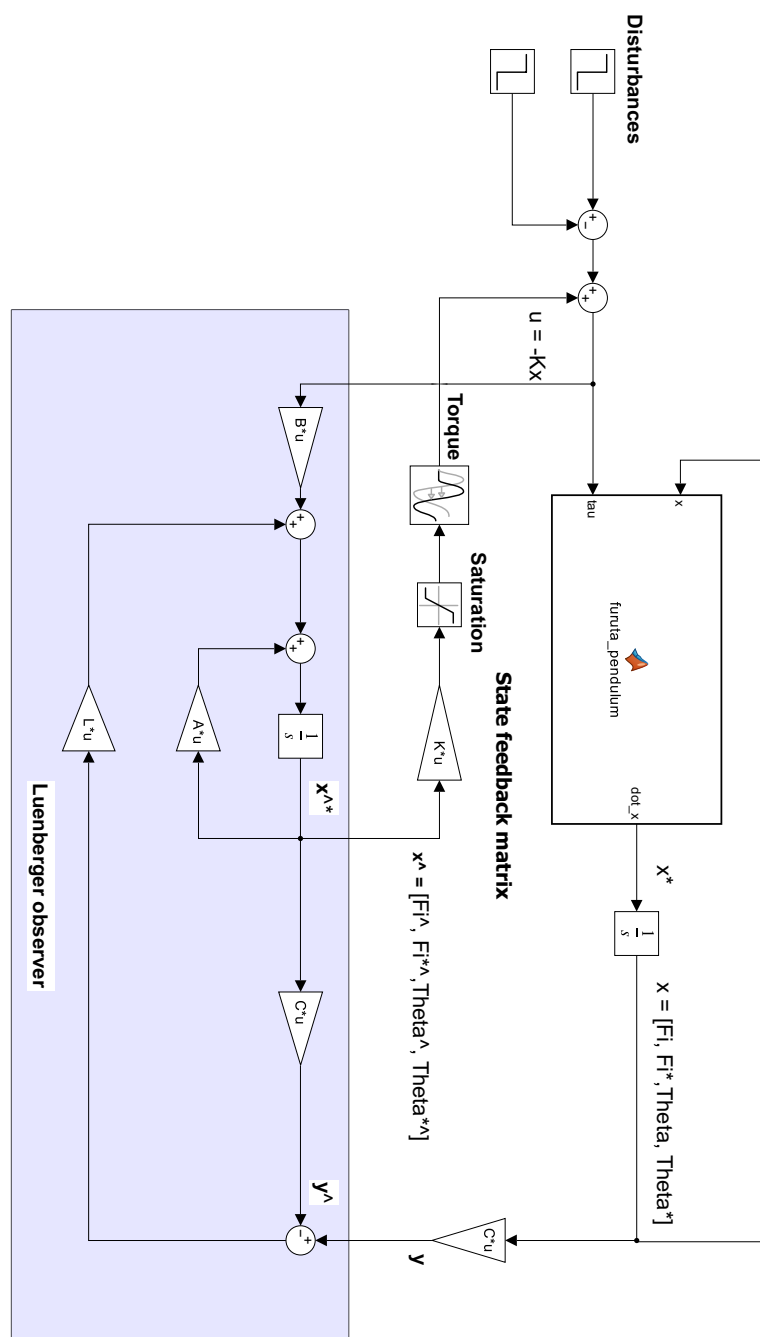
Obr. A.1: Schéma vektorového řízení



B Furuta Pendulum

B.1 Simulace řízení Furuta pendulum s estimátorem

Obr. B.1: Řízení matematického modelu Furuta pendulum s estimátorem



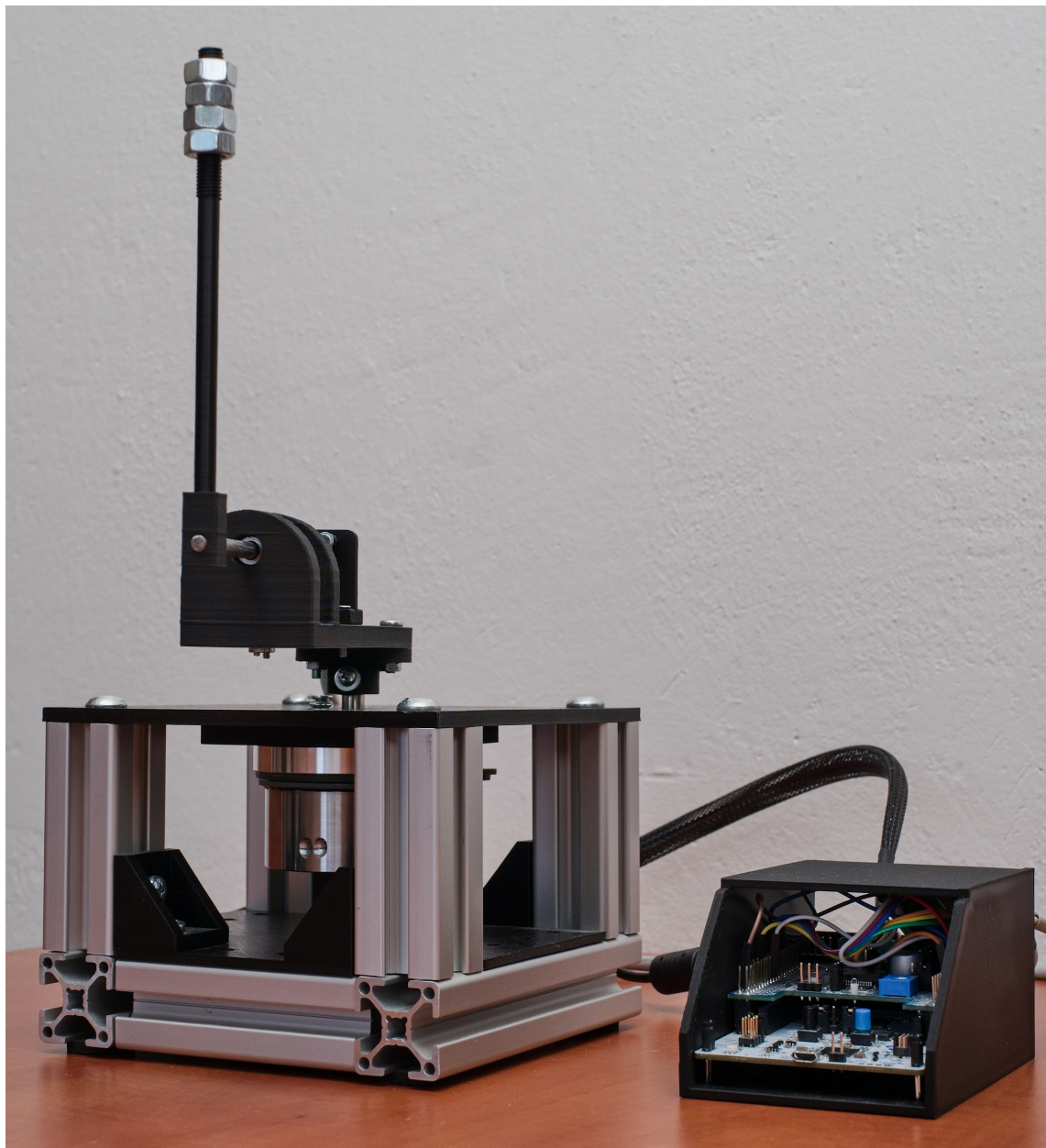
B.2 Implementace Luenbergerova pozorovatele

Výpis B.1: Implementace diskrétního Luenbergerova estimátoru v jazyce C

```
void StateSpaceObserver(float32_t u, float32_t y[2])
{
    //error
    float32_t e[2];
    // Model output:  $y_{hat} = C * x_{hat}$ 
    float32_t y_hat[2] = {0};
    y_hat[0] = x_states_hat[0]; //  $\phi$ 
    y_hat[1] = x_states_hat[2]; //  $\theta$ 
    //error -  $\Phi$  -  $\Theta$ 
    e[0] = y[0] - y_hat[0];
    e[1] = y[1] - y_hat[1];
    //Derivativ of states
    float32_t dx[4] = {0};
    // $x_{hat\_dot} = A*x_{hat} + B*u + L*e$ 
    for(int i = 0; i < 4; i++)
    {
        //  $A*x_{hat}$ 
        for(int j = 0; j < 4; j++)
        {
            dx[i] += A[i][j] * x_states_hat[j];
        }
        //  $B*u$ 
        dx[i] += B[i] * u;
        //  $L*e$ 
        dx[i] += L[i][0] * e[0] + L[i][1] * e[1];
    }
    //Integration(Sum - Euler)
    for(int i = 0; i < 4; i++)
    {
        x_states_hat[i] += dx[i] * dt;
    }
}
```

B.3 Sestrojené inverzní rotační kyvadlo

Obr. B.2: Laboratorní přípravek Furuta pendulum



C Obsah elektronické přílohy

Kvůli nutnosti udržet malou velikost odevzdaných souborů byla z projektové složky `bldc_cntrl` odstraněna složka `Middlewares`, která obsahuje knihovnu `ARM_CMSIS`. Složka lze překopírovat z projektové složky `Furuta_cntrl`.

```
/
├── BLDC.....Zdrojové soubory a modely pro řízení BLDC motoru
│   ├── bldc_cntrl.....Soubor .ioc a main.c
│   ├── CubeMX_For_Simulink.....Soubor .ioc pro Simulink
│   ├── BLDC_model_FOC.slx.....Matematický model BLDC motoru
│   ├── BLDC_model_constants.m.....Parametry BLDC motoru
│   └── FOC_STM32.slx.....Simulink kód pro FOC na STM32
├── Furuta Pendulum.....Podklady pro model a řízení Furuta pendulum
│   ├── Díly_3d_tisk
│   ├── Furuta_cntrl.....Soubor .ioc a main.c
│   ├── Furuta_Model_Control_Simulation.slx.....Model Furuta a estimátor
│   ├── Main.m.....Parametry Furuta pendulum a návrhové metody regulátoru
└── Skript_CubeMonitor_To_Matlab.mlx....Skript pro import dat ze souboru
```